FINAL YEAR PROJECT / DATA SOURCING PROTOCOL

Sourcing and Extracting Conversational Urdu Audio from YouTube

A step-by-step, legally grounded protocol for building a multi-speaker, overlap-rich Urdu speech collection at scale, ready for speaker diarization annotation and speaker-attributed transcription.

اردو گفتگو کے آڈیو کا حصول: مرحلہ وار طریقۂ کار

PROJECT
Speaker Diarization and Speaker-Attributed Transcription for Conversational Urdu

TEAM
Areeba Hasnain (lead)
Syed Ukkashah Ahmed
Ibrahim Johar Farooqi

SUPERVISION
Dr. Muhammad Yaseen Khan
Dr. Muhammad Rafi (co-supervisor)

VERSION
1.0 · 29 September 2026
Dept. of AI & Data Science
FAST-NUCES Karachi

Document control

Status	Draft for supervisor review. Collection must not start until the legal gate in Section 3.11 is signed.
Scope	Discovery, licence checking, download, conversion, screening, selection, splitting and hand-off of YouTube audio. Annotation and transcription are out of scope and start from the hand-off package in Section 8.
Companion files	Scripts `common.py` and `s01`–`s12`, `requirements.txt`, `keywords_seed.txt` (full listings in Appendix D).
Change policy	Any change to thresholds, tiers or legal position is logged with a date and the approver's name in the change log below, and the version number is increased.
Not legal advice	This protocol summarises published practice and publicly available legal information. It is not legal advice. Decisions marked "supervisor sign-off" must be made by the supervisors (and, where they decide it is needed, the university).

Version	Date	Change	Approved by
1.0	29 Sep 2026	First complete protocol.	

Contents

1 Read this first 3

2 What we are collecting, and in which formats 5

3 Legal and ethical framework 7

- 3.2 YouTube licences explained 7
- 3.5 Pakistani law 8
- 3.6 YouTube Terms of Service and downloading 9
- 3.8 The four release tiers 9
- 3.11 Legal gate: sign-off before collection 11

4 Precedents from the literature, and how they handled each requirement 12

5 Setting up the environment 17

6 The pipeline, step by step (Steps 0–11) 20

- Step 1 Keyword discovery 21
- Step 3 Metadata screening 22
- Step 5 Download and conversion 23
- Step 6 Automatic audio screening 24
- Step 7 Human triage 25
- Step 8 Excerpt selection 26
- Step 9 Splits and speaker linking 26
- Step 10 Hand-off package 27

7 Running at scale 28

8 Handing off to annotation and transcription 30

9 Checklists 32

10 References 33

Appendix A Seed keyword list 35

Appendix B Triage guide 36

Appendix C Glossary 37

Appendix D Script listings 38

1 Read this first

1.1 What this document is for

Recording conversations ourselves cannot produce enough hours to match, let alone exceed, the size of existing conversational Urdu resources. YouTube hosts thousands of hours of multi-party Urdu conversation: podcasts, interviews, panel discussions, talk shows and group vlogs, much of it with natural overlapping speech. This document turns that opportunity into a procedure that three students can run safely, repeatably and at scale.

It covers everything from the first keyword search to a clean, documented package of audio excerpts that the annotation and transcription stages can pick up without asking where any file came from, what licence it carries, or how it was processed. It does **not** cover annotation guidelines, transcription conventions or model training; those start from the hand-off package defined in Section 8.

1.2 How to use it

1. **Read Sections 1–4 once, all three of you, before touching any code.** Section 3 explains what we may and may not do; Section 4 shows how published datasets handled the same questions, which is what you will cite when reviewers ask.
2. **Complete the legal gate (Section 3.11)** with the supervisors. Nothing is downloaded before it is signed.
3. **Set up once (Section 5)**, then run the **pilot (Step 0)** on about ten videos end to end.
4. **Run Steps 1–10 in order**, in weekly batches (Section 7). Each step lists its owner, the command to run, what should come out, and a "check before moving on" list. Do not start a step until the previous step's checks pass.
5. **Run Step 11 every month** for as long as the corpus exists.

1.3 Who does what

The split below follows the individual tasks in Table II of the FYP proposal. It is a suggestion; change it if the team agrees, and record the change in the document control table.

Member	Proposal role	Responsibilities in this protocol
Ibrahim Johar Farooqi	Audio preprocessing, dataset partitions, validation of data and splits	Owns the pipeline machine and the provenance database. Runs Steps 3, 5, 6, 8, 9 and 10; maintains scripts; produces the weekly numbers (Section 7.6).
Syed Ukkashah Ahmed	CONVURL audit, annotation protocol, annotator agreement, documentation	Owns the triage guide (Appendix B) and the calibration round; defines channel-vetting criteria; aligns output formats and statistics with the CONVURL audit so the two corpora can be compared; drafts the datasheet.
Areeba Hasnain	Research coordination, literature, integration	Owns the legal gate and supervisor sign-offs, creator permission requests (Tier B), the takedown inbox, and the data statement drawing on Section 4.
All three	Shared	Keyword lists (Step 1), channel vetting (Step 2), triage (Step 7), speaker-link review (Step 9b), monthly maintenance review (Step 11).

NOT LEGAL ADVICE

This protocol explains copyright licences, platform terms and Pakistani law as far as they can be read from public sources, and shows how the research community has handled them. It cannot tell you what a court would decide. Wherever the text says *supervisor sign-off*, the decision belongs to Dr. Yaseen Khan and Dr. Rafi, who may in turn consult the university.

1.4 Conventions

- Commands are shown in `monospace` and are run from the `scripts/` folder with the Python environment active.
- **Callout boxes:** amber boxes are legal points, red boxes are things that can go badly wrong, green boxes are practical tips, and purple boxes point to a published precedent.
- Tiers are written A B C X and defined in Section 3.8.
- Numbers in square brackets such as [1] refer to Section 10; every entry has a clickable link.

2 What we are collecting, and in which formats

2.1 What counts as a usable recording

A recording is worth keeping when it gives the future models something the proposal needs: several real people talking to each other in Urdu, sometimes at the same time. The table is the checklist used in triage (Step 7).

Criterion	Keep	Reject or down-rank
Language	Urdu is the main language. Natural Urdu–English code-switching is welcome, since it is part of real conversational Urdu.	Mostly English, Punjabi, Pashto, Sindhi or another language (a separate regional subset can be decided later).
Interaction	Spontaneous conversation: podcasts, interviews, panels, talk shows, debates, group vlogs, classroom or seminar discussion.	Scripted drama, news bulletins read from a script, lectures or sermons by one person, recitation (naat, qawwali, mushaira), songs.
Speakers	At least two. **No upper limit**: large panels and crowded talk shows are wanted, because they are the hardest cases for diarization.	One speaker only; voice-over narration over footage.
Overlap	Any amount. Interruptions, back-channels (“haan”, “jee”) and cross-talk are valuable.	Heavily edited content where overlaps were cut out, or rapid montage editing.
Length	Source video of 8 minutes to about 4 hours; we cut 8–15 minute excerpts.	Shorts and clips under 8 minutes; multi-hour livestreams with long silences (manual check only).
Audio	Intelligible speech; room noise, phone-quality guests and studio audio are all fine and add variety.	Continuous background music under the speech, heavy distortion, speech that humans cannot follow.
People	Adults speaking in a public-facing role (hosts, guests, panellists, public figures).	Children speaking; private individuals disclosing personal, health, legal or family matters (Section 3.9).
Rights	Original uploader; licence recorded; fits a release tier.	Re-uploads of someone else’s show, compilations, content with inserted third-party clips.

2.2 Output formats at a glance

Every file the pipeline creates has a fixed format chosen for the later annotation, transcription and modelling stages. These choices follow the conventions used by pyannote.audio [23], by the diarization benchmarks cited in Section 4, and by the proposal’s preprocessing plan (16 kHz, standardised audio, recorded settings).

Artefact	Format	Where	Why
Original download	Exactly as served (usually Opus in `.webm`, or AAC in `.m4a`). Never modified.	`audio_raw/<channel>/<id>.*`	Evidence of what was obtained; lets us regenerate everything with better settings later.

Artefact	Format	Where	Why
Working copy	WAV, PCM signed 16-bit little-endian, 16 000 Hz, mono	`audio_wav16k/<id>.wav`	The standard input for pyannote, Whisper, WavLM and most toolkits; lossless and sample-exact for cutting.
Excerpt audio	Same WAV format, cut sample-exactly	`handoff/<ver>/audio/`	What annotators and models receive.
Machine diarization	RTTM (NIST Rich Transcription Time Marked)	`machine_rttm/`, `handoff/<ver>/rttm_machine/`	Standard diarization format; becomes the pre-annotation that humans correct.
Scoring regions	UEM (un-partitioned evaluation map)	`handoff/<ver>/uem/`	Tells scoring tools which part of each file is annotated.
Split lists	Plain text, one excerpt ID per line	`handoff/<ver>/lists/`	Channel-disjoint train/dev/test, used by pyannote.database and custom loaders.
Manifest	JSON Lines, one object per excerpt	`handoff/<ver>/manifest.jsonl`	Single source of truth for each excerpt: audio facts, checksum, source, licence, split, status.
Provenance	SQLite database with an append-only event log	`provenance.sqlite`	Every discovery, decision and file, with timestamps and names.

WHY 16 KHZ MONO AND NOT THE ORIGINAL QUALITY?

Speech models for diarization and recognition work at 16 kHz, and YouTube conversation audio has little useful speech energy above 8 kHz. Converting once, up front, removes a whole class of "wrong sample rate" bugs later. The untouched originals are kept, so a higher-bandwidth version can always be regenerated.

2.3 Planning numbers

Automatic sourcing is a steep funnel. The figures below are **planning assumptions** for a corpus of roughly 30–50 annotated hours (gold plus silver, as discussed for the annotation stage). Replace them with real ratios after the pilot and the first weekly batch.

Stage	Planning figure	Comment
Candidate video IDs discovered (Steps 1–2)	15 000–25 000	Keyword search over date windows, plus channel expansion.
Pass metadata screening (Step 3)	5 000–8 000	Length, genre words, made-for-kids, availability.
Downloaded (Step 5)	1 500–2 500 videos	Approved channels only, Creative Commons first.
Full screening (Step 6)	600–1 000 videos	Best 30–40% of the probe stage.
Accepted in triage (Step 7)	350–550 videos	Human decision.
Excerpt pool (Step 8)	500–800 excerpts, about 100–160 h	A pool 2–4 times larger than annotation capacity, so the annotation stage can pick balanced subsets.
Annotated later (not in this protocol)	30–50 h	Gold and silver tiers.

3 Legal and ethical framework

3.1 The four questions we must answer for every video

Using someone else's recording for research raises four separate questions. They have different answers, and a video can pass one and fail another.

Question	What governs it	Where it is handled
1. Copyright May we copy, cut and share the audio?	The licence the uploader chose on YouTube; Pakistani copyright law; any permission we obtain.	Sections 3.2–3.5, tiers in 3.8, Steps 3, 4 and 7.
2. Platform terms May we download from YouTube and use its API?	YouTube Terms of Service [16]; YouTube API Services Developer Policies [17].	Sections 3.6–3.7, Step 5, Section 7.5.
3. Privacy Is it fair to the people speaking?	Pakistani law (PECA 2016, draft data-protection bill); research ethics norms; common sense about who is speaking and about what.	Sections 3.5 and 3.9, Step 7.
4. Research ethics Has the institution approved it?	Supervisors, and the department or university if they require formal review.	Legal gate, Section 3.11.

3.2 YouTube licences explained

When uploading, a creator picks one of two licences. The choice is visible in the data returned by the YouTube Data API (`status.license`) and, on the watch page, in the expanded description.

Licence	API value	What it means for us
Standard YouTube Licence (the default; the large majority of videos)	`youtube`	All rights reserved by the creator. We may not redistribute the audio. Research use of a copy is a grey area (Sections 3.5–3.6). Such videos can only enter the corpus as C (reference-only) or, with written permission, B.
Creative Commons Attribution (CC BY; YouTube offers version 3.0)	`creativeCommon`	Anyone may copy, adapt (for example cut and resample) and redistribute, including commercially, provided they give credit. This is the only licence that lets us publish the audio itself without asking. Candidate for A. YODAS lists all of its YouTube-derived subsets as CC BY 3.0 for exactly this reason [5].

3.3 What CC BY requires of us

CC BY is permissive, but it has conditions [19]. For every Tier A excerpt we must:

1. **Credit the creator**: title of the video, channel name, link to the video, and a link to the licence. `s11_package.py` writes this automatically into `ATTRIBUTION.md`.
2. **Say what we changed**: excerpted, down-mixed to mono, resampled to 16 kHz. Also written automatically.
3. **Keep the audio under CC BY 3.0.** We cannot relicense someone else's audio. Our own annotations (RTTM, transcripts) are our work and can carry our chosen licence (Section 3.9).

4. **Not imply endorsement** by the creator, and not add technical restrictions (such as encryption) that stop people exercising the licence.

3.4 When a Creative Commons label cannot be trusted

A CC label is only valid if the uploader owned the rights in the first place. Uploaders sometimes tick the CC box on material they do not own. Treat a CC video as **not** Tier A if any of these is true:

- It is a re-upload or clip of a TV channel, news network, another podcast or another YouTuber (check the channel: does it publish its own shows, with consistent hosts and branding?).
- The API flag `contentDetails.licensedContent` is true, meaning the video has been claimed by a content partner. `s03` marks these `A-check` for manual review.
- It contains third-party music beds, film clips, songs or inserted segments from other shows.
- The channel is an aggregator or compilation channel.

EVIDENCE AT COLLECTION TIME

An uploader can later switch a video back to the standard licence. CC licences are generally understood not to be revocable for copies obtained while the licence applied, but you must be able to show what the licence was when you downloaded. Step 4 therefore records the API licence value with a timestamp, requests a Wayback Machine snapshot [22] of the watch page, and asks for a manual screenshot of the licence line for every Tier A video.

3.5 Pakistani law

Copyright

Copyright in Pakistan is governed by the Copyright Ordinance, 1962, as amended [20]. Section 57 lists acts that are not infringements, including fair dealing for research and private study, and for criticism or review. Two cautions follow from reading it:

- Section 57 is an **enumerated** list of permitted purposes and work types, not the open-ended, four-factor "fair use" test of the United States. Arguments like the one in VietMed [10] rely on US law and do not transfer automatically.
- As we read it, whether the research exception covers **copying recordings and film soundtracks to build a dataset**, and especially **publishing** parts of them, has not been settled. It can support internal research use of copies; it is not a safe basis for redistributing standard-licence audio. That is why standard-licence material is Tier C (reference-only) unless the owner gives written permission.

Privacy and data protection

As of 2026, Pakistan has not enacted a comprehensive data-protection law. A Personal Data Protection Bill has been drafted and approved by the federal cabinet but has not passed both houses of Parliament [21]. The Prevention of Electronic Crimes Act 2016 (amended in 2025) contains offences relating to misuse of personal information and identity [21]. Our position: **treat every voice as personal data anyway**, and apply data-minimisation and purpose-limitation as if the draft bill (or GDPR-style rules) applied. Section 3.9 turns this into concrete rules.

3.6 YouTube Terms of Service and downloading

YouTube's Terms of Service [16] restrict accessing or downloading content other than through YouTube's own features, unless YouTube and the rights holders permit it. A CC licence gives copyright permission from the creator; it does not, by itself, change YouTube's platform terms. This is the main **grey area** of the whole project.

How the field handles it: many widely cited speech datasets were built by downloading YouTube audio for research, including VoxCeleb [2]–[4], VoxConverse [1], YODAS [5], MSDWild [7] and GigaSpeech [8]. They typically limit use to research, state that copyright remains with the original owners, release identifiers and timestamps instead of audio where they do not hold rights, and honour removal requests (Section 4 gives the details and exact locations).

OUR RECORDED POSITION (TO BE SIGNED AT THE LEGAL GATE)

1. We download audio only, at a low request rate, only for non-commercial academic research, only from videos that passed metadata screening, and only after supervisor sign-off.
2. We never circumvent technical measures: no CAPTCHA solving, no bypassing sign-in or bot checks, no use of personal account cookies, no proxy rotation. If YouTube asks for verification, the run stops (Section 7.5).
3. We redistribute audio only where we hold a licence (Tier A) or written permission (Tier B). Everything else is released, if at all, as video IDs, timestamps and our annotations (Tier C), following VoxCeleb [2]–[4].
4. We delete audio of rejected videos (Section 7.3), keep audio access-controlled, and honour takedown requests within 7 days (Step 11).

3.7 YouTube API Services Developer Policies

We use the official YouTube Data API v3 [18] for discovery and metadata. It is quota-limited and comes with its own policies [17]:

- **Quota.** A new Google Cloud project gets 10 000 units per day. A search call costs 100 units; `videos.list`, `channels.list` and `playlistItems.list` cost 1 unit each for up to 50 items. Each team member uses their own project (Section 5.3). Do not create extra projects purely to multiply quota.
- **30-day storage rule.** The policies limit how long API data obtained without user authorisation may be stored: after 30 calendar days it must be refreshed or deleted [17]. We comply by re-checking every video monthly (`s03 --recheck`, Step 11), and by keeping, for videos that have disappeared, only the video ID and our own research records (decisions, timestamps, file checksums). The SQL for clearing copied titles and descriptions of vanished videos is given in Step 11.
- **API used for metadata only.** As we read the policies, the API may not be used to download or store copies of audiovisual content without YouTube's approval. Our scripts use the API only to find videos and read their metadata; downloading is a separate step and falls under Section 3.6.
- **Key security.** API keys are secrets: restrict them to the YouTube Data API in the Cloud console, keep them in environment variables, never commit them to Git.

3.8 The four release tiers

Every accepted video gets exactly one tier. The tier decides what may ever be published. Dev and test splits are filled from Tiers A and B first (Step 9), so the evaluation sets do not decay when videos vanish.

Tier	Condition	What may be released	Precedent
A **Open**	CC BY on YouTube, uploader is the original creator, no third-party content, evidence captured (Step 4).	Excerpt audio under CC BY 3.0 with attribution, plus our annotations.	YODAS [5]; MOSEL's per-item licence logic [9].
B **Permitted**	Standard licence, but the rights holder gave written permission that explicitly covers research use and redistribution of excerpts.	Excerpt audio under the permission's terms, plus annotations.	Standard practice for licensed corpora.
C **Reference-only**	Standard licence, no permission, passes every other check.	Video ID, start and end times, and our annotations. Audio stays internal for research.	VoxCeleb URL-plus-timestamp release [2]–[4].
X **Excluded**	Fails any rights, privacy or content rule, or subject of a takedown.	Nothing. Local audio deleted.	MSDWild removal-on-request [7].

EXPECT TIER A TO BE SMALL

Creative Commons YouTube content skews toward broadcast-style material such as news, educational talks and announcements rather than casual conversation; the CS-YODAS authors report this bias for the CC pool behind YODAS [12]. Conversational Urdu under CC BY will be scarce. Tier B (asking creators) is therefore the most valuable route to a redistributable test set.

3.9 Privacy and ethics rules

1. **Public-facing speakers only.** Prefer people speaking in a public role: hosts, invited guests, panellists, public figures. Street interviews and vlogs with bystanders are Tier C at best, and excluded if anyone discusses private matters.
2. **No children.** Videos flagged made-for-kids are rejected automatically (Step 3). Any audible child speaker found in triage means exclusion, or a second opinion if the child is incidental and very brief.
3. **No sensitive personal disclosures.** Exclude excerpts where a person (especially a private individual) discusses their health, sexuality, religious conversion, legal cases, abuse, family disputes or finances, or where accusations are made against private individuals.
4. **Pseudonymous speakers.** Speaker labels are per-excerpt codes (`spk00`, `spk01`...). We do not name speakers in released files, and we do not release cross-video identity links. The speaker embeddings in Step 9b are internal quality-control data, kept access-controlled and deleted once splits are final.
5. **Data minimisation.** Delete audio of rejected videos after screening (Section 7.3). Keep only what the corpus needs.
6. **Access control.** Audio lives on the pipeline machine and an access-restricted shared drive; never on a public link.
7. **Takedown.** A team inbox receives removal requests from rights holders or from people who appear in recordings. Requests are acted on within 7 days using `s12_maintenance.py takedown` and logged.
8. **Annotation licence (supervisor decision).** Options: CC BY 4.0, as VoxConverse and VoxCeleb chose [1]–[4]; or a non-commercial research licence (CC BY-NC 4.0), in the spirit of GigaSpeech's research-only access terms [8]. CC licences cannot add extra restrictions, so an acceptable-use request (no

speaker identification or voice cloning of real people) can only be stated as a non-binding request unless a custom research licence is used.

3.10 Asking creators for permission (Tier B)

A few cooperative channels can supply more redistributable conversational hours than the entire CC pool. Send the request from a university email address, copy a supervisor, and store the reply (PDF) in `licence_evidence/permissions/`. The reply must explicitly cover redistribution; "you can use it" is not enough.

```
Subject: Permission request: academic use of [Channel name] episodes in an Urdu speech research dataset

Assalam-o-Alaikum [Name],

We are final-year Artificial Intelligence students at FAST-NUCES Karachi, supervised by
Dr. Muhammad Yaseen Khan and Dr. Muhammad Rafi. Our project builds an open research dataset
for recognising "who spoke when" in natural Urdu conversation, a language that has very few
such resources.

Your episodes are exactly the kind of natural, multi-speaker Urdu conversation we need. We would
like your permission to:
  1. download the audio of the episodes listed below;
  2. cut 8-15 minute excerpts and label who speaks when, and what they say;
  3. publish these excerpts and labels in a free, non-commercial research dataset, with full
     credit to [Channel name] and a link to each original episode.

We will not use the audio for advertising or any commercial purpose, will not name speakers in
the data files, and will remove any episode immediately if you ask us to at any time.

Episodes: [titles and links]

A short written reply confirming (or limiting) this permission is all we need. We are happy to
answer any questions or to send you the finished dataset.

With thanks,
[Name], on behalf of Areeba Hasnain, Syed Ukkashah Ahmed and Ibrahim Johar Farooqi
FAST-NUCES Karachi  |  [university email]  |  cc: [supervisor]
```

3.11 Legal gate: sign-off before collection

All items must be ticked, and the page signed, before Step 1 runs at scale. The pilot (Step 0) may run once items 1–4 are done.

☐ Supervisors have read Sections 3 and 4 and approved this protocol (version and date noted below).

☐ Supervisors decided whether departmental or university ethics review is required; outcome recorded: _____

☐ The recorded position on downloading (Section 3.6) is approved as written, or amended here: _____

☐ Release tiers (Section 3.8) and the annotation licence choice (Section 3.9, item 8) are approved.

☐ A team takedown inbox exists and is monitored: _____

☐ Storage for audio is access-controlled; location recorded: _____

☐ API keys are restricted and stored only in environment variables; `.gitignore` excludes `corpus/` and any `.env` file.

☐ All three team members have read Sections 1–4 and Section 7.5.

Dr. Muhammad Yaseen Khan · signature / date	Dr. Muhammad Rafi · signature / date	Group leader (Areeba Hasnain) · signature / date

4 Precedents from the literature

4.1 How to read this section

Each entry gives the full citation, the venue, clickable links, and then how that work handled the requirements this protocol deals with: **sourcing**, **licensing and release**, **quality control**, and **what we adopt or do differently**. Section numbers were checked against the papers themselves, except where an entry says otherwise.

PAPER VERSUS DATASET PAGE

Many datasets state their legal terms on the dataset website or repository README, not in the paper. Where a statement comes from such a page, it is labelled **[dataset page]**. Cite the paper for methods and the page (with an access date) for licence terms. Do not claim a paper says something that only its website says.

4.2 Summary

Work	Venue	Main lesson for us
VoxConverse [1]	Interspeech 2020	Closest precedent: a diarization benchmark built from YouTube keyword search, targeting overlap; semi-automatic labelling; CC BY 4.0 annotations with copyright left with owners.
VoxCeleb 1/2 [2]–[4]	Interspeech 2017, 2018; *Computer Speech & Language* 2020	Canonical release model for material we do not own: YouTube IDs plus timestamps (our Tier C).
YODAS [5] (and v3 [6])	IEEE ASRU 2023 (v3: 2026 preprint)	CC-only collection at scale; keyword search plus channel expansion; resumable job database.
MSDWild [7]	Interspeech 2022	In-the-wild, multilingual diarization data from public videos; removal on owner request; link rot is real.
GigaSpeech [8]	Interspeech 2021	YouTube and podcast audio distributed under gated, non-commercial research terms.
MOSEL [9]	EMNLP 2024	Licence compliance is per item; one restrictive item changes a dataset's status.
VietMed [10]	LREC-COLING 2024	Explicit in-paper legal argument (US fair use) for YouTube-derived speech data.
Loquacious Set [11]; CS-YODAS [12]	arXiv preprints 2025, 2026	Never trust metadata language labels; the CC pool is biased toward broadcast-style speech.

4.3 VoxConverse — the closest precedent

J. S. Chung, J. Huh, A. Nagrani, T. Afouras, A. Zisserman, "Spot the Conversation: Speaker Diarisation in the Wild," *Proc. Interspeech 2020*, pp. 299–303. Paper: `isca-archive.org/interspeech_2020/chung20_interspeech.html` · DOI `10.21437/Interspeech.2020-2337` · arXiv `2007.01216` · Data: `github.com/joonson/voxconverse`

Requirement	How they handled it, and where
Sourcing	**§4.1, "Stage 1. Collection of videos":** YouTube keyword search starting from terms such as "panel debate" and "discussion", chosen to find videos where several people talk in turn or at once; duplicate URLs from different keywords removed; near-identical videos removed using tf-idf over YouTube's auto-generated subtitles. The resulting list ranged from US presidential debates and talk shows to documentaries.

Requirement	How they handled it, and where
Content	**§3, "Dataset description":** political debates, panel discussions, celebrity interviews, comedy news segments and talk shows. Average of 4–6 speakers per video with one video containing 21; average overlap about 3–4% of speech, but up to about 30% in individual videos (Table 2). This supports our "no upper speaker limit" rule and talk shows as a source.
Labelling	**§4:** automatic audio-visual diarization first, human verification second. **§4.1 "Discussion":** unlike VoxCeleb, a diarization dataset must label each recording in full; uncertain regions cannot simply be dropped. This is why we label whole excerpts.
Quality control	**§4.2, "Manual verification":** the authors verified early output themselves to learn failure modes and write guidelines (split segments at pauses over 0.25 s; laughter not assigned to speakers; short words such as "yes" count as speech; boundaries within 0.1 s). One hour from 15 videos was labelled twice independently, with about 1% DER between annotators. Labelling from scratch could take around ten times real time, while verifying machine output took around twice real time; pipeline thresholds were re-tuned on corrected videos over several rounds and verification got faster each time.
Licence and release	**[dataset page]** The GitHub README releases the dataset for research under CC BY 4.0 and states that copyright remains with the original owners of the videos; it also notes that debates and news were chosen to obtain lots of overlap. The Oxford project page links to downloadable dev and test WAV files.
We adopt	Keyword seeding with deduplication (Steps 1, 8); full labelling of every selected excerpt; machine pre-annotation before human correction (Step 6 output feeds annotation); their guideline values as a starting point for our annotation guide.
We do differently	Audio-only; 8–15 minute excerpts rather than whole videos; we do not host audio we hold no licence for (Tier C), because the Pakistani legal position (Section 3.5) is less clear than the UK context they worked in.

4.4 VoxCeleb 1 and 2 — the reference-only release model

[2] A. Nagrani, J. S. Chung, A. Zisserman, "VoxCeleb: a large-scale speaker identification dataset," *Proc. Interspeech 2017*. DOI `10.21437/Interspeech.2017-950` · arXiv `1706.08612`

[3] J. S. Chung, A. Nagrani, A. Zisserman, "VoxCeleb2: Deep speaker recognition," *Proc. Interspeech 2018*. DOI `10.21437/Interspeech.2018-1929` · arXiv `1806.05622`

[4] A. Nagrani, J. S. Chung, W. Xie, A. Zisserman, "VoxCeleb: Large-scale speaker verification in the wild," *Computer Speech & Language*, vol. 60, art. 101027, 2020. DOI `10.1016/j.csl.2019.101027` · Data: `robots.ox.ac.uk/~vgg/data/voxceleb/`

Requirement	How they handled it, and where
Sourcing	The journal paper [4] describes a multi-stage, fully automatic pipeline that starts from YouTube videos, confirms the visible speaker with an audio-visual synchronisation network, and confirms identity with face recognition.
Licence and release	**[dataset page]** The project page provides a URL for each YouTube video and timestamps for each utterance; the dataset is available for research under CC BY 4.0, and copyright stays with the owners of the videos.
We adopt	The Tier C release format: video ID, start and end times, and our own annotations, with a statement that copyright remains with the owners.
We do differently	Our unit is a continuous multi-speaker excerpt, not a single-speaker utterance; we also track the licence of every item and keep a redistributable core (Tiers A/B).

4.5 YODAS — Creative Commons collection at scale

[5] X. Li, S. Takamichi, T. Saeki, W. Chen, S. Shiota, S. Watanabe, "YODAS: YouTube-Oriented Dataset for Audio and Speech," *Proc. IEEE Automatic Speech Recognition and Understanding Workshop (ASRU) 2023*. DOI `10.1109/ASRU57964.2023.10389689` · arXiv `2406.00899` · Data: `huggingface.co/datasets/espnet/yodas`

[6] YODAS v3 (preprint, not yet peer reviewed), arXiv `2609.29448`, September 2026.

Requirement	How they handled it, and where
Licence rule	**§2, "Data Collection":** the first of two design requirements is that every video must carry a Creative Commons licence. **Table 1** lists all YODAS subsets as CC BY 3.0.
Discovery	**§2.1, "Keyword-based Client":** keywords extracted from multilingual Wikipedia dumps, with less common languages prioritised; searches restricted with YouTube's filters to (mostly) subtitled, CC-licensed videos; lower-ranked results reached by paging deeper.
Scaling	**§2.2, "Channel-based Client":** keyword search alone kept returning the same popular videos, so every discovered video's channel was crawled for more; videos on the same channel usually share licensing and subtitle characteristics.
Download and state	**§2.3:** audio stored as one channel at 24 kHz; language identification tools were not reliable enough for their purposes. **§2.4, "Master Node":** a PostgreSQL database tracks every keyword, channel and video as not-started, in-progress or done, so many workers can run without duplicating work.
Data quality	**§4.1:** YouTube subtitles were often misaligned or described scenes rather than speech, so alignment-based filtering was needed.
v3 update	The v3 preprint [6] keeps the CC-only restriction on search and deliberately favours recently uploaded videos, because YouTube's default ranking favours highly relevant, high-view-count videos.
We adopt	A separate CC-only discovery track (Step 1, `--track cc`); channel expansion (Step 2); a resumable state database (SQLite); newest-first ordering (`--order date`). We do not rely on YouTube subtitles at all.
We do differently	We use the official Data API for discovery instead of crawling search pages, and we add a human originality check, because a CC flag alone does not prove the uploader owned the content (Section 3.4).

4.6 MSDWild — in-the-wild diarization data and takedowns

[7] T. Liu, S. Fan, X. Xiang, H. Song, S. Lin, J. Sun, T. Han, S. Chen, B. Yao, S. Liu, Y. Wu, Y. Qian, K. Yu, "MSDWild: Multimodal Speaker Diarization Dataset in the Wild," *Proc. Interspeech 2022*, pp. 1476–1480. Paper: `isca-archive.org/interspeech_2022/liu22t_interspeech.html` · DOI `10.21437/Interspeech.2022-10466` · Data: `github.com/X-LANCE/MSDWILD`

Requirement	How they handled it, and where
Sourcing	**Abstract and introduction:** over 3 000 clips (about 80 hours) collected from public videos across many real-world scenarios and languages, restricted to naturally shot videos without heavy editing such as lens switching; a large share of naturally overlapped speech. Their related-work discussion also describes VoxConverse's keyword-based YouTube collection.
Release	Audio and video are both released.
Takedown and link rot	**[dataset page]** The README states the materials may be removed at any time at the original owner's request, and that a video archive was later uploaded because some source videos were no longer available online.
We adopt	The takedown clause and procedure (Step 11); preference for unedited, naturally recorded conversations; filling dev/test from redistributable tiers so evaluation data survives link rot.

4.7 GigaSpeech — gated research access

[8] G. Chen et al., "GigaSpeech: An Evolving, Multi-domain ASR Corpus with 10,000 Hours of Transcribed Audio," *Proc. Interspeech 2021*, pp. 3670–3674. arXiv `2106.06909` · Data: `huggingface.co/datasets/speechcolab/gigaspeech`

Requirement	How they handled it, and where
Sourcing	Transcribed audio collected from audiobooks, podcasts and YouTube, covering read and spontaneous speech across many topics.
Licence and release	**[dataset page]** The Hugging Face card states that SpeechColab does not own the copyright of the audio, and that access is granted through a gated request for non-commercial research and educational use under stated terms.

Requirement	How they handled it, and where
Lesson	A gated, research-only access model is an accepted option for material the curators do not own. MOSEL [9] later classed GigaSpeech as not open-source compliant precisely because it contains YouTube videos under restrictive licences, which shows how such choices affect a dataset's re-use.

4.8 MOSEL — licence compliance is per item

[9] M. Gaido, S. Papi, L. Bentivogli, et al., "MOSEL: 950,000 Hours of Speech Data for Open-Source Speech Foundation Model Training on EU Languages," *Proc. EMNLP 2024*. arXiv 2410.01036

Requirement	How they handled it, and where
Licence analysis	**§2, "Open Source Compliant Speech Data":** only freely accessible data under licences that allow derivative works and redistribution is included; Creative Commons data with NonCommercial, NoDerivatives or ShareAlike terms is excluded; and a dataset is excluded if it contains items under non-compliant licences, even when the dataset's own licence is permissive (GigaSpeech is their example).
We adopt	Licence recorded per video (Steps 3–4) and per excerpt (manifest); Tier A kept pure so it can be released under a single, clear licence.

4.9 VietMed — an explicit legal argument in the paper

[10] K. Le-Duc, "VietMed: A Dataset and Benchmark for Automatic Speech Recognition of Vietnamese in the Medical Domain," *Proc. LREC-COLING 2024*. arXiv 2404.05659

Requirement	How they handled it, and where
Legal reasoning	The paper's copyright discussion (check the exact section number in the published PDF before citing) works through the four US fair-use factors for YouTube-crawled audio: non-commercial research purpose, factual content, and short segments that do not compete with the originals or affect their viewership. It lists GigaSpeech, VoxCeleb and VoxLingua107 as comparable YouTube-derived research datasets.
Caution	This is a US-law argument. Pakistan uses enumerated fair dealing (Section 3.5). Use VietMed to show the field reasons about these questions openly, not as legal cover.

4.10 Supporting preprints

[11] "Loquacious Set: 25,000 Hours of Transcribed and Diverse English Speech Recognition Data for Research and Commercial Use," arXiv 2505.21578, 2025. · [12] "CS-YODAS: A Mined Dataset of In-the-Wild Code-Switched Speech," arXiv 2606.11514, 2026.

- **Loquacious Set** reports that up to half of YODAS's nominally English data was not English, and notes that GigaSpeech's YouTube licences were not checked for commercial use. Lesson: verify language with a language-ID model and with human ears (Steps 6–7); never trust channel or metadata language fields.
- **CS-YODAS** notes that YODAS's Creative Commons source biases it toward broadcast-style material such as news, educational talks and announcements. Lesson: expect few conversational Urdu CC videos, and plan for Tiers B and C.

Cite preprints as preprints; do not present them as peer-reviewed.

4.11 Documentation standards

[13] Our audit reference, CONVURL: N. Zaheer, A. A. Raza, M. Shabbir, "Conversations in the wild: Data collection, automatic generation and evaluation," *Computer Speech & Language*, vol. 89, 101699, 2025 (d

oi.org/10.1016/j.csl.2024.101699). Report our corpus statistics in the same units as the audit (hours, speakers per recording, overlap ratio, turn lengths) so the two are directly comparable.

[14] T. Gebru et al., "Datasheets for Datasets," *Communications of the ACM* 64(12), 2021 (doi.org/10.1145/3458723), and [15] E. M. Bender, B. Friedman, "Data Statements for Natural Language Processing," *TACL* 6, 2018 (doi.org/10.1162/tacl_a_00041) give the structure for the dataset documentation (Section 8.4).

4.12 Suggested wording for the data statement

DRAFT PARAGRAPH FOR THE REPORT OR PAPER

"Audio was sourced from publicly available YouTube videos, following established practice for in-the-wild speech corpora (VoxCeleb [2]–[4], VoxConverse [1], YODAS [5], MSDWild [7]). Every video's licence was recorded at collection time. Excerpts from videos published under CC BY by their original creators, or used with written permission, are distributed with attribution; for all other material we release only video identifiers, timestamps and our annotations, and copyright remains with the original owners. Speakers are pseudonymised, content involving minors or sensitive personal disclosures was excluded, and removal requests are honoured."

5 Setting up the environment

5.1 Hardware and storage

Item	Recommendation	Reason
Pipeline machine	One Linux machine (or WSL2) that holds the database and runs Steps 1–11	SQLite must have a single writer. Do not put the database on Google Drive or another sync folder: simultaneous writes corrupt it.
GPU	Any CUDA GPU with 8 GB or more (departmental workstation, or Colab/Kaggle sessions for Step 6)	Diarization, language ID and speaker embeddings. Steps 1–5 and 7–11 run on CPU.
Storage	250 GB or more of dedicated space (external SSD/HDD is fine)	Opus/AAC originals take roughly 55–60 MB per hour; 16 kHz WAV takes about 115 MB per hour. 2 000 videos of about 45 minutes is about 1 500 hours, so audio is deleted from rejected videos as you go (Section 7.3).
Backup	Weekly copy of `provenance.sqlite` and `licence_evidence/` to the access-controlled shared drive	The database and evidence are irreplaceable; audio can be re-downloaded while it still exists.

5.2 Software installation

```
# 1. system tools (Ubuntu / WSL2)
sudo apt update && sudo apt install -y ffmpeg git sqlite3 python3.11 python3.11-venv
ffmpeg -version && ffprobe -version        # both must print a version

# 2. project environment
git clone <your-private-repo> urdu-corpus && cd urdu-corpus
python3.11 -m venv .venv && source .venv/bin/activate
pip install -U pip
pip install -r scripts/requirements.txt    # see Appendix D
pip install -U yt-dlp                      # update yt-dlp weekly: YouTube changes often

# 3. record exact versions for reproducibility (commit this file)
pip freeze > environment.lock.txt && yt-dlp --version >> environment.lock.txt
```

GPU NOTES

Install the PyTorch build that matches your CUDA version from pytorch.org before `pip install -r. inaSpeechSegmenter` pulls in TensorFlow; if the two frameworks clash in one environment, run Step 6 in a separate environment that only has what `s06_auto_screen.py` imports. pyannote.audio changed its API between versions 3 and 4 (the token argument and the pipeline output object); the scripts handle both, but record the version you used.

5.3 YouTube Data API key (each team member)

1. Sign in to the Google Cloud Console with your university account and create a project named `urdu-corpus-<yourname>`.
2. Open *APIs & Services → Library*, find **YouTube Data API v3**, click *Enable*.
3. Open *Credentials → Create credentials → API key*. Then *Edit* the key: under *API restrictions* choose *Restrict key* and select only YouTube Data API v3. Save.

4. Store the key only as an environment variable on the pipeline machine: `export YT_API_KEY="…"` (for example in a `.env` file that is listed in `.gitignore`).
5. Check the quota page (*APIs & Services → YouTube Data API v3 → Quotas*): the default is 10 000 units per day, reset at midnight Pacific Time.

5.4 Hugging Face access for pyannote models

1. Create a Hugging Face account and an access token with *read* scope.
2. Open the model pages for the diarization pipeline you use (default in the scripts: `pyannote/speaker-diarization-3.1`, plus its segmentation dependency) and the embedding model `pyannote/wespeaker-voxceleb-resnet34-LM`, and accept the user conditions on each.
3. `export HF_TOKEN="…"`. To use a newer pipeline, set `DIAR_MODEL` to its name and record it; the model name is saved with every screening result.

5.5 Folder layout and naming

```
urdu-corpus/
  scripts/                  common.py, s01...s12, requirements.txt, keywords_seed.txt
  corpus/                   ($URDU_CORPUS_ROOT, never committed to Git)
    provenance.sqlite       the database (single writer: the pipeline machine)
    audio_raw/<channel_id>/<video_id>.webm|m4a    originals, never modified
    audio_wav16k/<video_id>.wav                   16 kHz mono PCM working copies
    machine_rttm/<video_id>.rttm                  full-video machine diarization
    licence_evidence/       screenshots, permissions/ (Tier B e-mails as PDF)
    speaker_embeddings/     internal QC only (Step 9b)
    handoff/<version>/      packages for annotation (Section 8)
    logs/
```

Identifier	Pattern	Example
Video	YouTube video ID (11 characters; may contain `-` and `_`)	`aB3-x_9QzLk`
Excerpt	`<video_id>__<start in ms, 9 digits>`	`aB3-x_9QzLk__000754120` (starts at 754.120 s)
Speaker (released files)	`spk00`, `spk01` … per excerpt	never a real name
Package version	`v<major>.<minor>`	`v0.1` (first hand-off), `v1.0` (FYP-I release)

Excerpt IDs are derived from the source and the start time, so re-running selection never creates two IDs for the same audio, and anyone can trace an excerpt back to its source position without the database.

5.6 The provenance database

`common.py` creates all tables automatically. Every script writes both its results and a row in the append-only `events` table (who, when, what, details). Never edit the database by hand except through the scripts or the documented SQL in this protocol.

Table	Holds
`keywords`	Each keyword × track × date window searched, with result counts (makes discovery resumable).
`channels`	Channel ID, title, uploads playlist, how it was found, vetting decision and note, expansion time.
`videos`	API metadata: channel, title, description (truncated), dates, duration, licence, partner-claim flag, made-for-kids flag, audio language, screening status and reason, preliminary tier.

Table	Holds
`licence_evidence`	Timestamped API licence value, Wayback snapshot URL, screenshot path, notes.
`downloads`	Original path, codec, sample rate, channels, bitrate, duration, WAV path, SHA-256 of both files, yt-dlp version, status.
`screening`	Probe and full results: speech/music/noise ratios, language fractions, speakers, overlap, maximum concurrency, speaker changes per minute, RTTM path, model name, score.
`triage`	Human decisions with reviewer name, tier and every checklist answer (Appendix B).
`excerpts`	Excerpt ID, source, start/end, hypothesised speakers and overlap, score, tier, split.
`events`	Append-only audit log of everything above.
`takedowns`	Removal requests and what was done.

6 The pipeline, step by step

The pipeline turns keywords into a packaged, documented set of excerpts. Each box below is one step; every step reads from and writes to the provenance database, so any step can be stopped and resumed.

0 · Pilot
Ten videos end to end; calibrate thresholds and timings.

1 · Discover
Keyword search over date windows; CC track and all-licence track.

2 · Channels
Vet channels by hand, then list every upload of approved ones.

3 · Metadata screen
Length, genre words, made-for-kids, licence, availability.

4 · Licence evidence
Timestamped licence record and Wayback snapshot for CC candidates.

5 · Download
Original audio stream unchanged; 16 kHz mono WAV copy; checksums.

6 · Auto-screen
Probe, then full: speech/music, language, speakers, overlap, score.

7 · Triage
People listen, check rights and privacy, assign a tier.

8 · Excerpts
Best 8–15 min windows, cut at pauses, capped per channel.

9 · Splits
Channel-disjoint train/dev/test; flag the same speaker across splits.

10 · Package
Excerpt WAVs, RTTM, UEM, lists, manifest, attribution, checksums.

11 · Maintain
Monthly licence and availability re-check; takedowns.

0 Pilot run

Owner: whole team · 2–3 days

Purpose. Prove that every script works on your machines, measure real timings and yields, and calibrate the thresholds before spending quota and storage at scale.

1. Put 5 keywords (from Appendix A) in `pilot_keywords.txt`. Run Step 1 with `--since 2024-01-01 --max-pages 1` on both tracks.
2. Run Steps 3 and 2 (vet about 10 channels), then Step 5 with `--limit 10 --tier any`.
3. Run Step 6 probe and full on those 10. Time both stages per audio hour.
4. All three members triage the same 10 videos independently (Step 7), compare answers, and fix the triage guide wherever you disagreed.
5. Run Steps 8–10 and open the package: listen to three excerpts, load one RTTM in Audacity (import as labels after converting) or in pyannote, and confirm the times line up with the audio.
6. Write the measured numbers (yield per stage, minutes of compute per audio hour, storage per hour) into the Section 2.3 table.

Check before moving on: every script ran; excerpt audio plays and RTTM times match speech; the team agrees on at least 8 of 10 triage decisions; the legal gate is fully signed.

1 Keyword discovery

Owner: all (keywords) · runner: pipeline machine

Purpose. Build a large list of candidate videos that are likely to contain multi-speaker Urdu conversation.

1A · BUILD THE KEYWORD LIST

- Start from Appendix A (`keywords_seed.txt`): Urdu-script, Roman-Urdu and English terms for conversational formats (podcast, interview, conversation, debate, panel, talk show, round table, Q&A, chit-chat).
- Add format × topic combinations after the pilot, for example پوڈکاسٹ کرکٹ, "urdu podcast technology", "panel discussion education pakistan". Topic variety gives speaker and vocabulary variety.
- Add city and region names (Karachi, Lahore, Peshawar, Quetta, Hyderabad, Multan) to reach regional accents.
- Do **not** add names of individual people. Search by format and topic, not by person.
- Keep the file under version control; every change is part of the method you will report.

1B · RUN DISCOVERY

```
python s01_discover.py --keywords keywords.txt --track cc  --since 2012-01-01
python s01_discover.py --keywords keywords.txt --track all --since 2012-01-01 --window-months 6 --
max-pages 3
```

What happens. For each keyword and each half-year publication window, the script calls `search.list` with `type=video`, `relevanceLanguage=ur`, `regionCode=PK`, newest-first ordering, and (for the CC track) `videoLicense=creativeCommon`. Splitting by date window matters: a single query only pages through a limited number of results, while many narrow windows each return fresh videos. Every video and channel found is inserted into the database with the keyword that found it.

Quota. Each page of up to 50 results costs 100 units, so one project can run about 100 pages per day. 40 keywords × 28 windows × 1–3 pages is 1 100–3 400 pages: plan 4–12 project-days, spread across the three keys. The script is resumable: when quota runs out it saves progress and exits cleanly; run it again the next day.

PRECEDENT

VoxConverse seeded its YouTube search with conversational format words and deduplicated across keywords [1, §4.1]; YODAS restricted search to CC-licensed videos [5, §2.1] and its v3 favours recent uploads to escape popularity bias [6].

Check before moving on: `SELECT COUNT(*) FROM videos;` grows each day; `SELECT keyword, SUM(n_results) FROM keywords GROUP BY keyword ORDER BY 2;` shows which keywords are useless (zero results everywhere) so you can replace them.

2 Channel vetting and expansion

Owner: all (vetting) · Ukkashah (criteria)

Purpose. Good conversational content clusters in channels (a podcast publishes dozens of similar episodes). Vetting a channel once saves vetting each of its videos, and listing a channel's uploads is about 100 times cheaper in quota than searching.

Order. Run Step 3 on the Step 1 results first (so licence counts are known), then vet, then expand, then run Step 3 again on the new videos.

```
python s02_channels.py pending                  # prints channel links with video and CC counts
python s02_channels.py vet UCxxxxxxxx approved "own weekly Urdu podcast, 2-4 hosts"
python s02_channels.py vet UCyyyyyyyy rejected "re-uploads TV talk show clips"
python s02_channels.py expand --max-videos 500
```

HOW TO VET A CHANNEL (2–3 MINUTES EACH)

1. Open the channel page. Does it publish its own shows, with recurring hosts, its own branding and consistent format? If it posts clips of other people's shows or compilations, **reject** (rights, Section 3.4).
2. Skim 3 recent video titles and thumbnails: conversational formats in Urdu? If mostly monologue, music, drama or news reading, **reject**.
3. Is it aimed at children? **Reject**.
4. Note in one line why you approved or rejected; the note is stored and becomes part of the method.

PRECEDENT

YODAS added channel-based crawling after finding that keyword search keeps returning the same popular videos, and observed that videos from one channel usually share licensing characteristics [5, §2.2].

Check before moving on: no approved channel is a re-upload or aggregator; `SELECT vetting, COUNT(*) FROM channels GROUP BY vetting;` shows progress.

3 Metadata screening

Owner: Ibrahim · ~1 quota unit per 50 videos

Purpose. Throw away obviously unsuitable videos before downloading anything, and record each video's licence.

```
python s03_screen_metadata.py
```

What happens. Videos are sent to `videos.list` in batches of 50 (parts `snippet`, `contentDetails`, `status`). Rules applied, in order:

Field	Rule	Outcome
`status.privacyStatus`	not `public`	reject (not_public)
`status.madeForKids`	true	reject (made_for_kids)
`snippet.liveBroadcastContent`	live / upcoming	defer, re-check later
`contentDetails.duration`	< 8 min / > 4 h	reject too_short / defer for manual check
title + first 500 chars of description	matches genre list (naat, qawwali, song, drama episode, khutba, mushaira…)	reject (excluded_genre_term)
not returned by the API	deleted, private or blocked	gone

Field	Rule	Outcome
`status.license` + `contentDetails.licensedContent`	CC and unclaimed / CC but partner-claimed / standard	tier_prelim `A?` / `A-check` / `C?`

The genre list lives in `common.py` (`EXCLUDE_TERMS`); extend it when triage keeps rejecting the same kind of video. Metadata language (`defaultAudioLanguage`) is stored but **not** used to reject, because creators often leave it wrong [11]. Licence changes between runs are logged automatically as `licence_changed` events.

Check before moving on: `SELECT meta_status, meta_reason, COUNT(*) FROM videos GROUP BY 1,2;` looks plausible; spot-check 10 rejected titles to make sure the genre list is not over-rejecting.

4 Licence evidence for Creative Commons candidates

Purpose. Freeze proof of the licence at the time of collection (Section 3.4).

```
python s04_licence_evidence.py --limit 200
```

1. For every CC candidate that passed Step 3, the script asks the Wayback Machine to snapshot the watch page, pausing 15 s between requests, and stores the snapshot link with the API licence value and a UTC timestamp.
2. If the Wayback Machine rate-limits (HTTP 429), the script stops; run it again later.
3. After triage confirms a video as Tier A, one team member opens the watch page, expands the description so the licence line is visible, takes a full-window screenshot, and saves it as `licence_evidence/<video_id>.png`.
4. For Tier B, save the permission e-mail thread as PDF in `licence_evidence/permissions/<channel_id>.pdf`.

Check before moving on: every Tier A candidate has at least one of: Wayback link or screenshot; ideally both.

5 Download and conversion

Owner: Ibrahim

Purpose. Obtain the audio exactly as served, and create a standard working copy.

```
python s05_download.py --limit 50              # Creative Commons candidates first
python s05_download.py --limit 50 --tier any   # then everything that passed Step 3
```

WHAT THE SCRIPT DOES FOR EACH VIDEO

1. **Download** with yt-dlp, audio only, no re-encoding:

```
yt-dlp -f "bestaudio/best" --no-playlist --write-info-json --no-overwrites \
       --retries 10 --fragment-retries 10 \
       --sleep-requests 1 --sleep-interval 5 --max-sleep-interval 20 \
       -o "audio_raw/<channel_id>/%(id)s.%(ext)s" "https://www.youtube.com/watch?v=<id>"
```

   The `.info.json` saved next to the audio records title, uploader, upload date and description at download time.

2. **Inspect** the original with ffprobe (codec, sample rate, channels, bitrate, duration) and store the values.

```
ffprobe -v error -select_streams a:0 -show_entries
stream=codec_name,sample_rate,channels,bit_rate:format=duration -of json <file>
```

3. **Convert** to the working format: first audio stream only, down-mixed to mono, resampled to 16 kHz, 16-bit PCM WAV.

```
ffmpeg -hide_banner -nostdin -y -i <original> -map 0:a:0 -vn -ac 1 -ar 16000 -c:a pcm_s16le au-
dio_wav16k/<id>.wav
```

4. **Checksum** both files (SHA-256) so any later corruption or accidental edit is detectable.

RATE LIMITS AND BLOCKS

The sleep options space requests out, and the script only downloads from approved channels by default. If YouTube responds with a sign-in or "confirm you are not a bot" message, or HTTP 429, the script stops the whole run. Do not work around it (Section 7.5).

Order of work. Download Tier A candidates first (they are scarce and most valuable), then standard-licence videos from approved channels in batches sized to your free disk space.

Check before moving on: `SELECT status, COUNT(*) FROM downloads GROUP BY status;` ; play two random WAVs; confirm with `soxi` or ffprobe that WAVs are 16000 Hz, 1 channel, 16-bit.

6 Automatic audio screening and ranking

Owner: Ibrahim · GPU

Purpose. Measure each recording so people only listen to the promising ones, and produce the machine diarization that later becomes pre-annotation.

```
python s06_auto_screen.py --stage probe --limit 100
python s06_auto_screen.py --stage full --min-score 0.45 --limit 40
```

TWO STAGES

- **Probe:** three 5-minute pieces (at 10%, 50% and 80% of the file) are joined into a temporary 15-minute clip and analysed. Cheap, and enough to rank.
- **Full:** the best probes (score ≥ `--min-score`) are analysed in full, and the full RTTM is kept in `machine_rttm/`.

MEASUREMENTS

Measure	Tool	How it is computed
Speech / music / noise ratios	inaSpeechSegmenter [25] (`vad_engine="smn"`)	Seconds labelled speech, music or noise, divided by the clip length. Music under speech is the most common reason to reject.
Urdu fraction	Whisper language ID [24] (`LID_MODEL`, default `small`)	30-second windows every 2 minutes, only where at least 70% is speech; share of windows whose top language is Urdu **or Hindi**. Spoken Urdu and Hindi are very close and Whisper often labels Urdu speech as Hindi, so the two are counted together and a human confirms Urdu in triage.

Measure	Tool	How it is computed
Speakers, overlap, concurrency, speaker changes	pyannote.audio [23] (`DIAR_MODEL`, default `pyannote/speaker-diarization-3.1`)	From the diarization: number of distinct speakers; overlapped time ÷ speech time; largest number of simultaneous speakers; speaker changes per minute of speech.

RANKING SCORE (0 TO 1)

```
score = 0.30 * min(overlap_ratio / 0.15, 1)       # 15 % overlap or more = full marks
      + 0.25 * min(log2(n_speakers) / log2(8), 1)  # 2 speakers = 0.33, 8 or more = 1.0
      + 0.25 * urdu_hindi_fraction
      + 0.20 * speech_ratio
      - 0.40 * music_ratio
# recordings with fewer than 2 detected speakers get score 0
```

More speakers never lower the score, so there is no upper speaker limit. The weights are a starting point: after the pilot, check that the videos you accepted in triage mostly have higher scores than the ones you rejected, and adjust if not.

COMPUTE PLANNING

Time the probe and full stages per audio hour during the pilot and use that to size batches. Run Step 6 on the departmental GPU or in Colab/Kaggle sessions with the `corpus/` folder mounted; results go straight into the database.

Check before moving on: open five full-stage RTTMs next to their audio and confirm the speaker count is roughly right; check that high-score videos sound like good conversations and low-score ones do not.

7 Human triage

Owner: Ukkashah (guide) · all three review

Purpose. People decide what enters the corpus and under which tier. The machine only ranks.

```
python s07_triage.py export --top 150 --out triage_batch_01.csv
# open in Google Sheets or Excel, divide rows, fill the columns, download as CSV
python s07_triage.py import triage_batch_01_done.csv
```

1. **Calibrate first.** All three review the same 20 videos, compare, and discuss every disagreement. Update Appendix B. Record the agreement rate on the `decision` column; repeat calibration if it is below 80%.
2. **Listen** for about 2 minutes per video: 40 seconds at the start of the conversation (after any intro), 40 seconds in the middle, and 40 seconds at the busiest part (the highest-overlap region in the RTTM if you have it open).
3. **Fill every column** using the definitions in Appendix B. Use `second_opinion` when unsure; another member decides.
4. **Tier decision:** A only if the licence is CC, the channel is the original creator, and there is no third-party content; B only with written permission on file; otherwise C; X for anything that fails a rule.

The import step validates values and blocks inconsistent rows (for example Tier A without `original_creator=yes`, or an accept with minors or private disclosures marked yes).

PRECEDENT

VoxConverse's authors verified the early material themselves to learn the failure modes before writing guidelines for others [1, §4.2]; our calibration round plays the same role.

Check before moving on: the import prints no errors; each reviewer's accept rate is in a similar range (a large gap means the guide is being read differently).

8 Excerpt selection

Owner: Ibrahim

Purpose. Turn accepted videos into manageable, information-dense excerpts for annotation.

```
python s08_select_excerpts.py --target-min 12 --min-min 8 --max-min 15 --per-video 2 --channel-cap-hours 3
```

1. Videos shorter than 15 minutes are taken whole; longer videos are scanned with a 12-minute window moved in 30-second steps.
2. Each window is scored from the machine diarization: 0.5 × overlap (saturating at 15%) + 0.3 × speaker count (log scale, no upper limit) + 0.2 × speech density.
3. The best windows are kept, at most two per video and at least one minute apart.
4. Each boundary is moved to the middle of the nearest pause of at least 0.4 s within 45 s, so no one is cut off mid-sentence.
5. Once a channel reaches `--channel-cap-hours` of excerpts, no more are taken from it. This keeps one prolific podcast from dominating the voices, topics and acoustics.

Check before moving on: listen to the first and last 5 seconds of ten excerpts (clean starts and ends); `SELECT channel_id, SUM(end_s-start_s)/3600 FROM excerpts GROUP BY 1 ORDER BY 2 DESC LIMIT 10;` shows no channel above the cap.

9 Channel-disjoint splits and speaker linking

Owner: Ibrahim · review: all

Purpose. Make sure test results measure performance on unseen speakers and recordings, as the proposal requires.

9A · ASSIGN SPLITS BY CHANNEL

```
python s09_assign_splits.py --test 0.10 --dev 0.10 --seed 2026
```

- Whole channels go into one split, so a podcast host never appears in both training and test data.
- Dev and test are filled first from channels with Tier A or B excerpts, so the evaluation sets can be redistributed and will not decay as YouTube videos disappear.
- Assignments are frozen once made; new channels top up the split furthest below target. Record the seed.

9B · FIND THE SAME PERSON ACROSS SPLITS

```
python s10_speaker_link.py --threshold 0.55 --out speaker_link_review.csv
```

- Guests, anchors and politicians appear on many channels. For every hypothesised speaker in every excerpt, the script builds a voice embedding (WeSpeaker ResNet34 through pyannote [26]) from up to 30 s of their clean, non-overlapped turns, and lists pairs from *different* splits whose cosine similarity is above the threshold.
- Two people listen to each flagged pair and fill `same_person`. For confirmed matches, either move the smaller channel into the same split as the larger (then rerun 9a for the rest), or drop the excerpt from test/dev.
- Calibrate the threshold in the pilot using a few known same-person and different-person pairs.

Check before moving on: split hours are close to targets; no unresolved confirmed matches remain; the review CSV is saved in `logs/`.

10 Build the hand-off package

Owner: Ibrahim · receivers: annotation stage

```
python s11_package.py --version v0.1
```

Creates `handoff/v0.1/` with sample-exact excerpt WAVs, per-excerpt and per-split machine RTTMs (speakers renamed `spk00`...), UEM files, split lists, a `database.yml` for pyannote.database, `manifest.jsonl`, `sources.csv`, `public_manifest.csv`, `ATTRIBUTION.md` and `checksums.sha256`. Section 8 specifies every file.

Check before moving on: `sha256sum -c checksums.sha256` passes inside the package; every excerpt in `lists/*.lst` has audio, RTTM, UEM and a manifest line; `public_manifest.csv` marks only Tier A/B as redistributable.

11 Monthly maintenance and takedowns

Owner: Areeba (inbox) · Ibrahim (runs)

```
python s03_screen_metadata.py --recheck     # refresh all API metadata (30-day rule, Section 3.7)
python s12_maintenance.py report            # excerpts whose source vanished or changed licence

# removal request received:
python s12_maintenance.py takedown --request-id TD-2026-001 --video <video_id> \
       --requester "channel owner (e-mail on file)"
python s11_package.py --version v0.2        # rebuild and redistribute the package
```

- **Vanished videos.** Tier C excerpts whose source is gone can no longer be fetched by outside users; consider replacing them in the next package. Clear copied API text for them:

  ```
  UPDATE videos SET title=NULL, description=NULL WHERE meta_status='gone';
  ```

- **Licence changes.** For Tier A items whose licence was later changed, keep the collection-time evidence and raise each case with the supervisors.
- **Takedowns** are completed within 7 days, logged, and followed by a new package version.

7 Running at scale

7.1 Weekly cadence

Work in weekly batches so problems surface early and disk space stays under control. A suggested first six weeks:

Week	Work	Output at week end
0	Legal gate signed; environment set up; pilot (Step 0); triage calibration.	Signed gate; pilot numbers in Section 2.3.
1	Step 1 on both tracks with all three API keys; Step 3 on results; first channel vetting round.	Several thousand candidates; vetted channel list.
2	Step 2 expansion; Step 3 again; Step 4 evidence; Step 5 batch 1 (Tier A first); Step 6 probe.	First downloads screened.
3	Step 6 full on best probes; Step 7 triage batch 1; permission e-mails to the best standard-licence channels.	First accepted videos with tiers.
4	Step 5 batch 2; Step 6; Step 7 batch 2; Step 8 selection.	Excerpt pool, first version.
5	Step 9 splits and speaker linking; Step 10 package `v0.1` ; annotation pilot can begin.	Hand-off package v0.1.
6 onward	Top-up cycles (Steps 1–10) aimed at gaps: under-represented formats, regions or speaker counts; Step 11 monthly.	v0.2, v0.3...

7.2 Quota budgeting

Call	Units	Items per call	Typical use
`search.list` (Step 1)	100	up to 50 results	The expensive one. About 100 pages per project per day.
`videos.list` (Step 3, Step 11)	1	up to 50 videos	10 000 videos cost about 200 units.
`channels.list` (Step 2)	1	up to 50 channels	Negligible.
`playlistItems.list` (Step 2)	1	up to 50 uploads	A 500-video channel costs 10 units.

A good daily split per key: about 9 000 units of search (90 pages) and 1 000 units kept back for metadata and channel work. Use each member's key on different keyword files so work is not duplicated. If you need more, Google's quota-extension form exists, but it requires describing the use case honestly and may be refused; the three default quotas are enough for an FYP-scale corpus.

7.3 Storage and data minimisation

Delete audio you will never use. This saves space and is also the privacy rule in Section 3.9. Run after each triage import (keeps all database rows; removes only audio files):

```
python - <<'EOF'
import os, sqlite3
con = sqlite3.connect(os.environ["URDU_CORPUS_ROOT"] + "/provenance.sqlite")
rows = con.execute("""
  SELECT d.video_id, d.orig_path, d.wav_path FROM downloads d
  WHERE d.status = 'ok' AND (
    d.video_id IN (SELECT video_id FROM triage WHERE decision = 'reject')
    OR d.video_id IN (SELECT video_id FROM screening WHERE stage = 'probe' AND score < 0.30))
""").fetchall()
for vid, orig, wav in rows:
    for p in (orig, wav):
```

```
        if p and os.path.exists(p):
            os.remove(p)
    con.execute("UPDATE downloads SET status = 'audio_deleted' WHERE video_id = ?", (vid,))
    con.execute("INSERT INTO events VALUES (datetime('now'), 'cleanup', 'video', ?, 'audio_deleted', '')",
(vid,))
con.commit(); print(len(rows), "videos cleaned")
EOF
```

Keep originals and WAVs of accepted videos until the corpus is final. Tier A originals are kept permanently alongside their licence evidence.

7.4 Monitoring queries

```
-- the funnel, one line per stage
SELECT 'discovered', COUNT(*) FROM videos
UNION ALL SELECT 'metadata pass', COUNT(*) FROM videos WHERE meta_status='pass'
UNION ALL SELECT 'downloaded', COUNT(*) FROM downloads WHERE status IN ('ok','audio_deleted')
UNION ALL SELECT 'probe screened', COUNT(*) FROM screening WHERE stage='probe'
UNION ALL SELECT 'full screened', COUNT(*) FROM screening WHERE stage='full'
UNION ALL SELECT 'accepted', COUNT(DISTINCT video_id) FROM triage WHERE decision='accept'
UNION ALL SELECT 'excerpts', COUNT(*) FROM excerpts WHERE tier != 'X';

-- hours by tier and split
SELECT tier, split, COUNT(*) n, ROUND(SUM(end_s-start_s)/3600.0, 2) hours
FROM excerpts GROUP BY tier, split ORDER BY tier, split;

-- speaker-count and overlap profile of the excerpt pool (machine estimates)
SELECT n_speakers_hyp, COUNT(*) n, ROUND(AVG(overlap_ratio_hyp)*100, 1) avg_overlap_pct
FROM excerpts GROUP BY n_speakers_hyp ORDER BY n_speakers_hyp;

-- reviewer consistency
SELECT reviewer, COUNT(*) n, ROUND(AVG(decision='accept')*100) accept_pct FROM triage GROUP BY reviewer;
```

7.5 When YouTube blocks you

STOP, DO NOT CIRCUMVENT

If yt-dlp reports that YouTube wants you to sign in or confirm you are not a bot, or returns HTTP 429, `s05_download.py` stops by itself. Then:

1. Stop downloading for at least 24 hours.
2. Update yt-dlp (`pip install -U yt-dlp`); many failures are simply an outdated extractor.
3. Resume with smaller batches and longer sleep intervals.
4. If blocks persist, tell the supervisors and pause. Do **not** pass personal account cookies to yt-dlp, rotate IP addresses or proxies, use CAPTCHA-solving services, or run from many machines to spread the load. Those steps would move the project from a grey area into clear breach of the Terms of Service (Section 3.6).

7.6 Weekly report (to supervisors)

One short table every week, generated from the queries in Section 7.4: the funnel; hours by tier and split; speaker-count and overlap profile; number of channels; the largest channel's share of hours; triage agreement and accept rate per reviewer; permission requests sent and granted; takedowns received. These same numbers become the corpus statistics in the final report.

8 Handing off to annotation and transcription

8.1 What the annotation stage receives

```
handoff/v0.1/
  audio/<excerpt_id>.wav            16 kHz, mono, PCM s16le, sample-exact cut from the working copy
  rttm_machine/<excerpt_id>.rttm    machine diarization for that excerpt (pre-annotation, NOT ground
truth)
  rttm_machine/{train,dev,test}.rttm  the same, concatenated per split
  uem/<excerpt_id>.uem              region to annotate and score: the whole excerpt
  uem/{train,dev,test}.uem
  lists/{train,dev,test}.lst        excerpt IDs, one per line, channel-disjoint
  database.yml                      pyannote.database protocol
UrduYT.SpeakerDiarization.MachinePreannotation
  manifest.jsonl                    one JSON object per excerpt (below)
  sources.csv                       excerpt -> video, channel, title, offsets, licence, evidence link
  public_manifest.csv               what may be redistributed (Tier A/B) vs reference-only (Tier C)
  ATTRIBUTION.md                    CC BY credits and permissions
  checksums.sha256                  verify with: sha256sum -c checksums.sha256
```

8.2 File format specifications

RTTM

One line per speaker turn, ten space-separated fields; times in seconds with millisecond precision, relative to the start of the excerpt:

```
SPEAKER <excerpt_id> 1 <onset> <duration> <NA> <NA> <speaker> <NA> <NA>
SPEAKER aB3-x_9QzLk__000754120 1 12.480 3.215 <NA> <NA> spk01 <NA> <NA>
```

Overlapping speech appears as two turns whose time spans intersect. Speaker labels are local to each excerpt: `spk01` in one excerpt is unrelated to `spk01` in another.

UEM

```
<excerpt_id> 1 <start> <end>
aB3-x_9QzLk__000754120 1 0.000 721.244
```

Manifest (one JSON object per line)

Field	Meaning
`excerpt_id`, `split`, `tier`	Identity, channel-disjoint split, release tier.
`audio`, `sample_rate`, `channels`, `sample_format`, `num_samples`, `duration_s`, `sha256`	Exact audio facts; loaders should assert these.
`source.video_id`, `url`, `channel_id`, `channel_title`, `title`, `published_at`	Where the audio came from.
`source.offset_start_s`, `offset_end_s`	Position of the excerpt inside the source video; also what Tier C users need to recreate it.
`source.api_license`, `licence_evidence`	Licence at collection time and the Wayback link.

Field	Meaning
`machine_preannotation.*`	RTTM path, model name, hypothesised speaker count and overlap ratio.
`status.diarization_gold`, `status.transcription`	Flags the annotation and transcription stages switch to `true` when done.

Loading with pyannote.database

```
export PYANNOTE_DATABASE_CONFIG=/path/to/handoff/v0.1/database.yml
python -c "from pyannote.database import registry; \
p = registry.get_protocol('UrduYT.SpeakerDiarization.MachinePreannotation'); \
print(sum(1 for _ in p.test()))"
```

8.3 Machine pre-annotation: useful, but not truth

- Correcting machine output is far faster than labelling from scratch; VoxConverse reports roughly twice real time for verification against around ten times for labelling from scratch [1, §4.2].
- Pre-annotation can bias annotators toward the machine's answer. The annotation stage should label a small subset from scratch and compare, and should never evaluate a model against uncorrected machine RTTMs.
- The machine speaker count and overlap in the manifest are estimates for planning and balancing only; report corpus statistics from the corrected annotations.

8.4 What to record for the dataset documentation

Following the datasheet [14] and data statement [15] structures, the sourcing stage must be able to supply: motivation and funding; the full keyword list and date windows; channel-vetting criteria and counts; screening thresholds and model versions; triage guide and agreement figures; the funnel numbers; tier and licence breakdown; split method and seed; speaker-linking threshold and outcomes; exclusions for privacy; the takedown policy; and known biases (for example, public-figure and broadcast-style speech is over-represented relative to everyday conversation). Everything listed is already in the database, the logs or this protocol.

9 Checklists

Before every weekly batch

☐ yt-dlp updated; free disk space greater than the batch size × 175 MB per audio hour.

☐ Database backed up (Section 5.1).

☐ Keyword file changes committed to Git.

☐ No open takedown requests older than 7 days.

After every batch

☐ Each step's "check before moving on" list is done.

☐ Audio of rejected videos deleted (Section 7.3).

☐ Weekly report sent (Section 7.6).

Before building a package

☐ Every Tier A excerpt has licence evidence (Wayback link and/or screenshot); every Tier B excerpt has a permission PDF.

☐ Speaker-link review finished; no confirmed cross-split matches remain.

☐ Split hours checked against targets; seed recorded.

☐ Package version number increased; checksums verify.

Before anything is made public

☐ Supervisors approved the release and the annotation licence.

☐ Only files listed as redistributable in `public_manifest.csv` include audio; Tier C is IDs, timestamps and annotations only.

☐ `ATTRIBUTION.md`, the datasheet, the takedown contact and a statement that copyright remains with the original owners (for Tier C) are included.

☐ No real speaker names, no speaker embeddings, no internal database in the public release.

☐ Step 11 re-check run within the last 30 days.

10 References

Links verified on 29 September 2026. Venue abbreviations: Interspeech (Annual Conference of the International Speech Communication Association), ASRU (IEEE Automatic Speech Recognition and Understanding Workshop), EMNLP (Conference on Empirical Methods in Natural Language Processing), LREC-COLING (Joint International Conference on Computational Linguistics, Language Resources and Evaluation), TACL (Transactions of the Association for Computational Linguistics).

Datasets and papers

[1] J. S. Chung, J. Huh, A. Nagrani, T. Afouras, A. Zisserman. Spot the Conversation: Speaker Diarisation in the Wild. *Interspeech 2020*, pp. 299–303. https://www.isca-archive.org/interspeech_2020/chung20_interspeech.html · https://doi.org/10.21437/Interspeech.2020-2337 · arXiv: https://arxiv.org/abs/2007.01216 · dataset page: https://github.com/joonson/voxconverse, https://www.robots.ox.ac.uk/~vgg/data/voxconverse/

[2] A. Nagrani, J. S. Chung, A. Zisserman. VoxCeleb: A Large-Scale Speaker Identification Dataset. *Interspeech 2017*. https://doi.org/10.21437/Interspeech.2017-950 · arXiv: https://arxiv.org/abs/1706.08612

[3] J. S. Chung, A. Nagrani, A. Zisserman. VoxCeleb2: Deep Speaker Recognition. *Interspeech 2018*. https://doi.org/10.21437/Interspeech.2018-1929 · arXiv: https://arxiv.org/abs/1806.05622

[4] A. Nagrani, J. S. Chung, W. Xie, A. Zisserman. VoxCeleb: Large-Scale Speaker Verification in the Wild. *Computer Speech & Language* 60, 101027, 2020. https://doi.org/10.1016/j.csl.2019.101027 · dataset page: https://www.robots.ox.ac.uk/~vgg/data/voxceleb/

[5] X. Li, S. Takamichi, T. Saeki, W. Chen, S. Shiota, S. Watanabe. YODAS: YouTube-Oriented Dataset for Audio and Speech. *IEEE ASRU 2023*. https://doi.org/10.1109/ASRU57964.2023.10389689 · arXiv: https://arxiv.org/abs/2406.00899 · data: https://huggingface.co/datasets/espnet/yodas

[6] YODAS v3 (preprint, September 2026). arXiv: https://arxiv.org/abs/2609.29448

[7] T. Liu, S. Fan, X. Xiang, H. Song, S. Lin, J. Sun, T. Han, S. Chen, B. Yao, S. Liu, Y. Wu, Y. Qian, K. Yu. MSDWild: Multi-modal Speaker Diarization Dataset in the Wild. *Interspeech 2022*, pp. 1476–1480. https://www.isca-archive.org/interspeech_2022/liu22t_interspeech.html · https://doi.org/10.21437/Interspeech.2022-10466 · dataset page: https://github.com/X-LANCE/MSDWILD

[8] G. Chen et al. GigaSpeech: An Evolving, Multi-domain ASR Corpus with 10,000 Hours of Transcribed Audio. *Interspeech 2021*, pp. 3670–3674. arXiv: https://arxiv.org/abs/2106.06909 · dataset card: https://huggingface.co/datasets/speechcolab/gigaspeech

[9] M. Gaido, S. Papi, L. Bentivogli, et al. MOSEL: 950,000 Hours of Speech Data for Open-Source Speech Foundation Model Training on EU Languages. *EMNLP 2024*. arXiv: https://arxiv.org/abs/2410.01036

[10] K. Le-Duc. VietMed: A Dataset and Benchmark for Automatic Speech Recognition of Vietnamese in the Medical Domain. *LREC-COLING 2024*. arXiv: https://arxiv.org/abs/2404.05659

[11] Loquacious Set: 25,000 Hours of Transcribed and Diverse English Speech Recognition Data for Research and Commercial Use (preprint, 2025). arXiv: https://arxiv.org/abs/2505.21578

[12] CS-YODAS: A Mined Dataset of In-the-Wild Code-Switched Speech (preprint, 2026). arXiv: https://arxiv.org/abs/2606.11514

[13] N. Zaheer, A. A. Raza, M. Shabbir. Conversations in the Wild: Data Collection, Automatic Generation and Evaluation. *Computer Speech & Language* 89, 101699, 2025. https://doi.org/10.1016/j.csl.2024.101699

[14] T. Gebru et al. Datasheets for Datasets. *Communications of the ACM* 64(12), 2021. https://doi.org/10.1145/3458723

[15] E. M. Bender, B. Friedman. Data Statements for Natural Language Processing: Toward Mitigating System Bias and Enabling Better Science. *TACL* 6, 2018. https://doi.org/10.1162/tacl_a_00041

Platform terms, licences and law

[16] YouTube Terms of Service. https://www.youtube.com/t/terms

[17] YouTube API Services — Developer Policies. https://developers.google.com/youtube/terms/developer-policies

[18] YouTube Data API v3 reference: search.list https://developers.google.com/youtube/v3/docs/search/list · videos https://developers.google.com/youtube/v3/docs/videos · playlistItems.list https://developers.google.com/youtube/v3/docs/playlistItems/list · quota costs https://developers.google.com/youtube/v3/determine_quota_cost

[19] Creative Commons Attribution 3.0 Unported, legal code: https://creativecommons.org/licenses/by/3.0/legalcode · CC BY 4.0: https://creativecommons.org/licenses/by/4.0/

[20] The Copyright Ordinance, 1962 (Pakistan), as amended. Pakistan Code: https://pakistancode.gov.pk/pdffiles/administrator5d255d1f44b50b65d6a3e87a2e0d3fee.pdf · WIPO Lex: https://www.wipo.int/wipolex/en/legislation/details/3482

[21] Data protection status in Pakistan: DLA Piper, Data Protection Laws of the World — Pakistan https://www.dlapiperdataprotection.com/index.html?t=law&c=PK · Chambers, Data Protection & Privacy 2026 — Pakistan https://practiceguides.chambers.com/practice-guides/data-protection-privacy-2026/pakistan

[22] Internet Archive Wayback Machine (Save Page Now). https://web.archive.org/

Tools

[23] H. Bredin. pyannote.audio 2.1 Speaker Diarization Pipeline: Principle, Benchmark, and Recipe. *Interspeech 2023*. Code: https://github.com/pyannote/pyannote-audio · model: https://huggingface.co/pyannote/speaker-diarization-3.1 · pyannote.database: https://github.com/pyannote/pyannote-database

[24] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, I. Sutskever. Robust Speech Recognition via Large-Scale Weak Supervision. *ICML 2023*, PMLR 202, pp. 28492–28518. https://proceedings.mlr.press/v202/radford23a.html · code: https://github.com/openai/whisper

[25] inaSpeechSegmenter (speech / music / noise segmentation, INA). https://github.com/ina-foss/inaSpeechSegmenter

[26] WeSpeaker speaker embeddings (ResNet34 model distributed through pyannote). https://huggingface.co/pyannote/wespeaker-voxceleb-resnet34-LM

[27] yt-dlp. https://github.com/yt-dlp/yt-dlp

[28] FFmpeg and ffprobe. https://ffmpeg.org/

A Seed keyword list

This is the content of `keywords_seed.txt`. Extend it with format × topic and format × city combinations after the pilot (Step 1a). Avoid personal names.

Urdu script	Roman Urdu	English
پوڈکاسٹ اردو پوڈکاسٹ انٹرویو خصوصی انٹرویو گفتگو خصوصی گفتگو مکالمہ مباحثہ مذاکرہ بحث مباحثہ ٹاک شو پینل ڈسکشن گول میز تبادلہ خیال سوال و جواب گپ شپ بیٹھک نشست	urdu podcast guftagu khas guftagu mubahisa gup shup baithak sawal jawab	pakistani podcast urdu interview urdu talk show panel discussion urdu roundtable discussion pakistan debate urdu urdu discussion university seminar urdu student podcast pakistan family vlog urdu

B Triage guide

Owner: Syed Ukkashah Ahmed. Revise after every calibration round and record the change in the document control table.

Listening protocol (about 2 minutes per video)

1. Skip any intro music or sponsor segment. Listen for 40 s at the start of the conversation.
2. Jump to the middle and listen for 40 s.
3. Jump to the busiest-looking part (many turns or overlaps in the RTTM, or a lively section on the YouTube progress bar) and listen for 40 s.
4. Glance at the channel page if you are unsure whether it is the original creator.

Column definitions

Column	Values	Answer yes when…
`original_creator`	yes / no / unsure	The channel produced this content itself (own hosts, own branding). A clip from a TV channel posted by someone else is **no**.
`conversational`	yes / no / unsure	Two or more people genuinely talk to each other, unscripted. A lecture with one question at the end is **no**.
`urdu_dominant`	yes / no / unsure	Most speech is Urdu (code-switching with English is fine).
`music_bed`	yes / no / unsure	Music plays underneath speech for long stretches (intro and outro jingles alone do not count).
`inserted_clips`	yes / no / unsure	The video embeds other people's clips, songs, news footage or ads with speech.
`minors`	yes / no / unsure	A child speaks at any point.
`private_info`	yes / no / unsure	A private individual discloses personal health, family, legal, financial or similar matters, or is accused of something.
`sensitive`	yes / no / unsure	Content that could harm the people in it if widely redistributed (for example, testimony about abuse or persecution). Flag for a second opinion.
`format_type`	podcast / talk_show / panel / interview / debate / vlog_group / classroom / other	Pick the closest; used to balance the corpus.
`decision`	accept / reject / second_opinion	Accept only if conversational = yes, urdu_dominant = yes, music_bed = no, minors = no, private_info = no.
`tier`	A / B / C / X	A: CC licence and original_creator = yes and inserted_clips = no. B: written permission on file. C: otherwise acceptable. X: rejected.
`notes`	free text	Anything unusual: number of speakers you heard, audio issues, regional accent, notable overlap.

C Glossary

Term	Meaning
Diarization	Deciding who spoke when in a recording.
Overlap ratio	Time during which two or more people speak at once, divided by total speech time.
Maximum concurrency	The largest number of people speaking simultaneously at any moment.
Excerpt	A continuous 8–15 minute section cut from a source video; the unit of annotation.
Channel-disjoint split	Train, dev and test sets that never share a YouTube channel, and therefore rarely share speakers.
Speaker linking	Finding the same person across different excerpts by comparing voice embeddings.
Voice embedding	A vector summarising a speaker's voice; similar voices give vectors with high cosine similarity.
RTTM	Rich Transcription Time Marked: the standard text format for diarization output and reference labels.
UEM	Un-partitioned Evaluation Map: which time ranges of each file are annotated and scored.
LID	Language identification.
Pre-annotation	Machine output given to annotators to correct, rather than labelling from zero.
Tier A / B / C / X	Release tiers: open (CC BY), permitted (written permission), reference-only (IDs and timestamps), excluded.
Quota unit	The YouTube Data API's cost measure; default allowance 10 000 units per project per day.
Provenance	The recorded history of where each item came from and what was done to it.
Link rot	Online sources disappearing over time, which breaks reference-only datasets.

D Script listings

These are the companion scripts delivered with this document. They were written for this protocol and checked for syntax; the offline steps (triage import, excerpt selection, splits, packaging, take-down) were run end to end on synthetic data. Steps that call YouTube, the Wayback Machine or GPU models (1–6, 9b) must be verified during the pilot (Step 0) on your own machines, because they depend on network access, API keys, model licences and library versions that could not be exercised here. Run everything from the `scripts/` folder.

D.1 `requirements.txt`

```
# Core (Steps 1-5, 7-9a, 11-12) - CPU is enough
google-api-python-client>=2.100
isodate>=0.6
requests>=2.31
yt-dlp            # keep it current: pip install -U yt-dlp   (YouTube changes often)
soundfile>=0.12
numpy>=1.24
# Screening + speaker linking (Steps 6, 9b) - GPU strongly recommended
torch
openai-whisper
inaSpeechSegmenter   # pulls in TensorFlow
pyannote.audio       # gated models: accept terms on Hugging Face and set HF_TOKEN
# System packages (not pip): ffmpeg + ffprobe  ->  sudo apt install ffmpeg   (or conda install -c conda-forge ffm-
peg)
```

D.2 `common.py`

```
"""common.py - shared configuration, paths and provenance-database helpers.

Every other script imports this file. Nothing here talks to YouTube.
Set these environment variables before running anything:
    URDU_CORPUS_ROOT   folder where all data lives (default: ./corpus)
    YT_API_KEY         YouTube Data API v3 key (one per team member / Google Cloud project)
    HF_TOKEN           Hugging Face token (needed for gated pyannote models)
"""
import json
import os
import sqlite3
from datetime import datetime, timezone
from pathlib import Path

# ---------------------------------------------------------------- paths
ROOT = Path(os.environ.get("URDU_CORPUS_ROOT", "./corpus")).resolve()
DB_PATH = ROOT / "provenance.sqlite"
RAW_DIR = ROOT / "audio_raw"            # untouched originals exactly as downloaded (read-only)
WAV_DIR = ROOT / "audio_wav16k"         # full-length 16 kHz mono 16-bit PCM working copies
RTTM_DIR = ROOT / "machine_rttm"        # machine diarization hypotheses for full videos
PROBE_DIR = ROOT / "tmp_probes"         # temporary probe clips for stage-1 screening
EVIDENCE_DIR = ROOT / "licence_evidence"
EMB_DIR = ROOT / "speaker_embeddings"
HANDOFF_DIR = ROOT / "handoff"          # packaged excerpts for the annotation stage
LOG_DIR = ROOT / "logs"

YT_API_KEY = os.environ.get("YT_API_KEY")
HF_TOKEN = os.environ.get("HF_TOKEN")

# ---------------------------------------------------------------- tunable thresholds
MIN_DURATION_S = 8 * 60           # shorter videos rarely contain usable 8-15 min conversations
MAX_DURATION_S = 4 * 60 * 60      # very long streams are mostly dead air; review manually if needed
TARGET_SR = 16000                 # sample rate of all working audio

EXCLUDE_TERMS = [                 # genre words that almost always mean music, scripted or monologue content
    "naat", "qawwali", "song", "lyrics", "drama episode", "full movie", "audiobook",
    "khutba", "bayan", "tilawat", "mushaira",
    "مشاعرہ", "تلاوت", "خطبہ", "ڈرامہ", "گانا", "قوالی", "نعت",
]

for d in (ROOT, RAW_DIR, WAV_DIR, RTTM_DIR, PROBE_DIR, EVIDENCE_DIR, EMB_DIR, HANDOFF_DIR, LOG_DIR):
```

```
    d.mkdir(parents=True, exist_ok=True)

SCHEMA = """
CREATE TABLE IF NOT EXISTS keywords(
    keyword TEXT, track TEXT, window_start TEXT, window_end TEXT,
    pages INTEGER, n_results INTEGER, done_at TEXT,
    PRIMARY KEY (keyword, track, window_start));
CREATE TABLE IF NOT EXISTS channels(
    channel_id TEXT PRIMARY KEY, title TEXT, uploads_playlist TEXT,
    discovered_via TEXT, discovered_at TEXT, expanded_at TEXT,
    vetting TEXT DEFAULT 'pending', vetting_note TEXT);
CREATE TABLE IF NOT EXISTS videos(
    video_id TEXT PRIMARY KEY, channel_id TEXT, discovered_via TEXT, discovered_at TEXT,
    title TEXT, description TEXT, published_at TEXT, duration_s REAL,
    license TEXT, licensed_content INTEGER, made_for_kids INTEGER,
    default_audio_lang TEXT, live TEXT, privacy TEXT,
    meta_checked_at TEXT, meta_status TEXT, meta_reason TEXT, tier_prelim TEXT);
CREATE TABLE IF NOT EXISTS licence_evidence(
    video_id TEXT, checked_at TEXT, api_license TEXT, wayback_url TEXT,
    screenshot_path TEXT, note TEXT);
CREATE TABLE IF NOT EXISTS downloads(
    video_id TEXT PRIMARY KEY, downloaded_at TEXT, orig_path TEXT, orig_codec TEXT,
    orig_sr INTEGER, orig_channels INTEGER, orig_bitrate INTEGER, duration_s REAL,
    wav_path TEXT, sha256_orig TEXT, sha256_wav TEXT, ytdlp_version TEXT,
    status TEXT, error TEXT);
CREATE TABLE IF NOT EXISTS screening(
    video_id TEXT, stage TEXT, screened_at TEXT,
    speech_ratio REAL, music_ratio REAL, noise_ratio REAL,
    urdu_hindi_frac REAL, english_frac REAL, other_lang_frac REAL, lid_windows INTEGER,
    n_speakers INTEGER, overlap_ratio REAL, max_concurrent INTEGER, changes_per_min REAL,
    rttm_path TEXT, diar_model TEXT, score REAL,
    PRIMARY KEY (video_id, stage));
CREATE TABLE IF NOT EXISTS triage(
    video_id TEXT, reviewer TEXT, reviewed_at TEXT, decision TEXT, tier TEXT,
    original_creator TEXT, conversational TEXT, urdu_dominant TEXT, music_bed TEXT,
    inserted_clips TEXT, minors TEXT, private_info TEXT, sensitive TEXT,
    format_type TEXT, notes TEXT);
CREATE TABLE IF NOT EXISTS excerpts(
    excerpt_id TEXT PRIMARY KEY, video_id TEXT, channel_id TEXT, start_s REAL, end_s REAL,
    n_speakers_hyp INTEGER, overlap_ratio_hyp REAL, score REAL,
    tier TEXT, split TEXT, created_at TEXT);
CREATE TABLE IF NOT EXISTS events(
    ts TEXT, actor TEXT, entity TEXT, entity_id TEXT, action TEXT, detail TEXT);
CREATE TABLE IF NOT EXISTS takedowns(
    request_id TEXT PRIMARY KEY, received_at TEXT, requester TEXT, video_id TEXT,
    action_taken TEXT, completed_at TEXT);
"""


def now() -> str:
    """UTC timestamp in ISO-8601, used for every *_at column."""
    return datetime.now(timezone.utc).isoformat(timespec="seconds")


def db() -> sqlite3.Connection:
    con = sqlite3.connect(DB_PATH)
    con.row_factory = sqlite3.Row
    con.executescript(SCHEMA)
    return con


def log_event(con, entity, entity_id, action, detail="", actor=None):
    """Append-only audit log. Never delete rows from this table."""
    actor = actor or os.environ.get("USER", "unknown")
    if not isinstance(detail, str):
        detail = json.dumps(detail, ensure_ascii=False)
    con.execute("INSERT INTO events VALUES (?,?,?,?,?,?)",
                (now(), actor, entity, entity_id, action, detail))


def youtube_client():
    """Authenticated YouTube Data API v3 client (API key only, public data only)."""
    if not YT_API_KEY:
        raise SystemExit("Set YT_API_KEY first (see Section 5.3 of the protocol).")
    from googleapiclient.discovery import build
    return build("youtube", "v3", developerKey=YT_API_KEY, cache_discovery=False)


def is_quota_error(err) -> bool:
```

```
    return "quotaExceeded" in str(err) or "dailyLimitExceeded" in str(err)


def read_rttm(path):
    """Return a list of (start, end, speaker) tuples from an RTTM file."""
    segs = []
    with open(path, encoding="utf-8") as f:
        for line in f:
            p = line.split()
            if len(p) >= 8 and p[0] == "SPEAKER":
                start, dur = float(p[3]), float(p[4])
                segs.append((start, start + dur, p[7]))
    return sorted(segs)


def write_rttm(path, uri, segs):
    with open(path, "w", encoding="utf-8") as f:
        for s, e, spk in sorted(segs):
            f.write(f"SPEAKER {uri} 1 {s:.3f} {e - s:.3f} <NA> <NA> {spk} <NA> <NA>\n")


def speech_stats(segs):
    """Speech time, overlapped time and max simultaneous speakers from diarization segments."""
    events = []
    for s, e, _ in segs:
        events += [(s, 1), (e, -1)]
    events.sort()                    # at equal times, ends (-1) sort before starts (+1)
    active, prev, speech, overlap, max_c = 0, None, 0.0, 0.0, 0
    for t, d in events:
        if prev is not None and active > 0:
            speech += t - prev
            if active > 1:
                overlap += t - prev
        active += d
        max_c = max(max_c, active)
        prev = t
    return speech, overlap, max_c


def speaker_changes(segs):
    ordered = sorted(segs)
    return sum(1 for a, b in zip(ordered, ordered[1:]) if a[2] != b[2])


def speech_union(segs):
    """Merge all speaker segments into non-overlapping speech intervals."""
    merged = []
    for s, e, _ in sorted(segs):
        if merged and s <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], e)
        else:
            merged.append([s, e])
    return merged
```

D.3 `s01_discover.py`

```
"""s01_discover.py - Step 1: keyword discovery through the YouTube Data API.

Runs every keyword over a series of publication-date windows so that each query
returns fresh results (a single query only ever pages through a few hundred hits).

    python s01_discover.py --keywords keywords.txt --track cc  --since 2012-01-01
    python s01_discover.py --keywords keywords.txt --track all --since 2012-01-01

--track cc   -> only Creative Commons videos (candidates for Tier A)
--track all  -> every licence (candidates for Tier B / Tier C)
Cost: 100 quota units per results page. Default quota: 10,000 units/day/project.
"""
import argparse
from datetime import date, datetime

from googleapiclient.errors import HttpError

from common import db, is_quota_error, log_event, now, youtube_client


def month_windows(since: date, until: date, months: int):
```

```
    start = since
    while start < until:
        m = start.month - 1 + months
        end = date(start.year + m // 12, m % 12 + 1, 1)
        yield start, min(end, until)
        start = end


def rfc3339(d: date) -> str:
    return datetime(d.year, d.month, d.day).strftime("%Y-%m-%dT00:00:00Z")


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--keywords", required=True, help="UTF-8 text file, one keyword per line")
    ap.add_argument("--track", choices=["cc", "all"], required=True)
    ap.add_argument("--since", default="2012-01-01")
    ap.add_argument("--until", default=date.today().isoformat())
    ap.add_argument("--window-months", type=int, default=6)
    ap.add_argument("--max-pages", type=int, default=3, help="pages of 50 results per window")
    ap.add_argument("--order", default="date", choices=["date", "relevance", "viewCount"])
    args = ap.parse_args()

    yt, con = youtube_client(), db()
    keywords = [k.strip() for k in open(args.keywords, encoding="utf-8")
                if k.strip() and not k.startswith("#")]
    since, until = date.fromisoformat(args.since), date.fromisoformat(args.until)

    for kw in keywords:
        for w_start, w_end in month_windows(since, until, args.window_months):
            done = con.execute("SELECT 1 FROM keywords WHERE keyword=? AND track=? AND window_start=?",
                               (kw, args.track, w_start.isoformat())).fetchone()
            if done:
                continue                                # resumable: skip finished windows
            params = dict(part="snippet", q=kw, type="video", maxResults=50,
                          relevanceLanguage="ur", regionCode="PK", order=args.order,
                          videoDuration="any",
                          publishedAfter=rfc3339(w_start), publishedBefore=rfc3339(w_end))
            if args.track == "cc":
                params["videoLicense"] = "creativeCommon"
            n, pages, token = 0, 0, None
            try:
                while pages < args.max_pages:
                    if token:
                        params["pageToken"] = token
                    resp = yt.search().list(**params).execute()
                    pages += 1
                    for item in resp.get("items", []):
                        vid = item["id"]["videoId"]
                        ch = item["snippet"]["channelId"]
                        con.execute("INSERT OR IGNORE INTO videos(video_id, channel_id, discovered_via,"
                                    " discovered_at, title, published_at) VALUES (?,?,?,?,?,?)",
                                    (vid, ch, f"kw:{kw}|{args.track}", now(),
                                     item["snippet"]["title"], item["snippet"]["publishedAt"]))
                        con.execute("INSERT OR IGNORE INTO channels(channel_id, title, discovered_via,"
                                    " discovered_at) VALUES (?,?,?,?)",
                                    (ch, item["snippet"]["channelTitle"], f"kw:{kw}", now()))
                        n += 1
                    token = resp.get("nextPageToken")
                    if not token:
                        break
            except HttpError as err:
                con.commit()
                if is_quota_error(err):
                    print("Daily quota exhausted - rerun tomorrow; progress is saved.")
                    return
                raise
            con.execute("INSERT OR REPLACE INTO keywords VALUES (?,?,?,?,?,?,?)",
                        (kw, args.track, w_start.isoformat(), w_end.isoformat(), pages, n, now()))
            log_event(con, "keyword", kw, "searched",
                      {"track": args.track, "window": [str(w_start), str(w_end)], "results": n})
            con.commit()
            print(f"{kw!r} {w_start}..{w_end} [{args.track}] -> {n} results")


if __name__ == "__main__":
    main()
```

D.4 `s02_channels.py`

```
"""s02_channels.py - Step 2: channel vetting and channel expansion.

    python s02_channels.py pending                      # list channels waiting for a human decision
    python s02_channels.py vet UCxxxx approved "independent Urdu podcast, own content"
    python s02_channels.py vet UCyyyy rejected "re-uploads TV clips"
    python s02_channels.py expand --max-videos 500      # list uploads of every approved channel

Expansion uses channels.list + playlistItems.list (1 quota unit per 50 videos),
which is ~100x cheaper than search.list.
"""
import argparse

from googleapiclient.errors import HttpError

from common import db, is_quota_error, log_event, now, youtube_client


def pending(con):
    rows = con.execute("""SELECT c.channel_id, c.title, COUNT(v.video_id) AS n,
                              SUM(v.license='creativeCommon') AS n_cc
                          FROM channels c LEFT JOIN videos v ON v.channel_id = c.channel_id
                          WHERE c.vetting = 'pending' GROUP BY c.channel_id ORDER BY n DESC""").fetchall()
    for r in rows:
        print(f"{r['channel_id']}  videos={r['n']:<4} cc={r['n_cc'] or 0:<4} "
              f"https://www.youtube.com/channel/{r['channel_id']}  {r['title']}")
    print(f"\n{len(rows)} channels pending vetting")


def vet(con, channel_id, decision, note):
    con.execute("UPDATE channels SET vetting=?, vetting_note=? WHERE channel_id=?",
                (decision, note, channel_id))
    log_event(con, "channel", channel_id, f"vetted:{decision}", note)
    con.commit()


def expand(con, max_videos):
    yt = youtube_client()
    todo = [r["channel_id"] for r in con.execute(
        "SELECT channel_id FROM channels WHERE vetting='approved' AND expanded_at IS NULL")]
    try:
        for i in range(0, len(todo), 50):                       # channels.list takes up to 50 ids
            batch = todo[i:i + 50]
            resp = yt.channels().list(part="contentDetails,snippet", id=",".join(batch),
                                      maxResults=50).execute()
            for ch in resp.get("items", []):
                uploads = ch["contentDetails"]["relatedPlaylists"]["uploads"]
                con.execute("UPDATE channels SET uploads_playlist=?, title=? WHERE channel_id=?",
                            (uploads, ch["snippet"]["title"], ch["id"]))
                n, token = 0, None
                while n < max_videos:
                    page = yt.playlistItems().list(part="contentDetails", playlistId=uploads,
                                                   maxResults=50, pageToken=token).execute()
                    for it in page.get("items", []):
                        con.execute("INSERT OR IGNORE INTO videos(video_id, channel_id, discovered_via,"
                                    " discovered_at) VALUES (?,?,?,?)",
                                    (it["contentDetails"]["videoId"], ch["id"],
                                     f"channel:{ch['id']}", now()))
                        n += 1
                    token = page.get("nextPageToken")
                    if not token:
                        break
                con.execute("UPDATE channels SET expanded_at=? WHERE channel_id=?", (now(), ch["id"]))
                log_event(con, "channel", ch["id"], "expanded", {"videos_listed": n})
                con.commit()
                print(f"{ch['snippet']['title']}: {n} uploads listed")
    except HttpError as err:
        con.commit()
        if is_quota_error(err):
            print("Daily quota exhausted - rerun tomorrow; progress is saved.")
            return
        raise


def main():
    ap = argparse.ArgumentParser()
    sub = ap.add_subparsers(dest="cmd", required=True)
```

```
    sub.add_parser("pending")
    v = sub.add_parser("vet")
    v.add_argument("channel_id")
    v.add_argument("decision", choices=["approved", "rejected", "pending"])
    v.add_argument("note", nargs="?", default="")
    e = sub.add_parser("expand")
    e.add_argument("--max-videos", type=int, default=500)
    args = ap.parse_args()
    con = db()
    if args.cmd == "pending":
        pending(con)
    elif args.cmd == "vet":
        vet(con, args.channel_id, args.decision, args.note)
    else:
        expand(con, args.max_videos)


if __name__ == "__main__":
    main()
```

D.5 `s03_screen_metadata.py`

```
"""s03_screen_metadata.py - Step 3: metadata screening before any audio is downloaded.

    python s03_screen_metadata.py            # checks every video not yet checked
    python s03_screen_metadata.py --recheck  # re-checks everything (monthly refresh, see Step 11)

Uses videos.list (1 quota unit per 50 videos). Writes meta_status = pass / reject / gone / defer
and a preliminary licence tier:  A? (Creative Commons), A-check (CC but flagged as
partner-claimed content), C? (standard YouTube licence).
"""
import argparse
import re

import isodate
from googleapiclient.errors import HttpError

from common import (EXCLUDE_TERMS, MAX_DURATION_S, MIN_DURATION_S, db, is_quota_error,
                    log_event, now, youtube_client)

EXCLUDE_RE = re.compile("|".join(re.escape(t) for t in EXCLUDE_TERMS), re.IGNORECASE)


def decide(item):
    sn, cd, st = item["snippet"], item["contentDetails"], item["status"]
    dur = isodate.parse_duration(cd["duration"]).total_seconds() if cd.get("duration") else 0
    text = f"{sn.get('title', '')} {sn.get('description', '')[:500]}"
    if st.get("privacyStatus") != "public":
        return dur, "reject", "not_public"
    if st.get("madeForKids"):
        return dur, "reject", "made_for_kids"
    if sn.get("liveBroadcastContent") in ("live", "upcoming"):
        return dur, "defer", "live_or_upcoming"
    if dur < MIN_DURATION_S:
        return dur, "reject", "too_short"
    if dur > MAX_DURATION_S:
        return dur, "defer", "very_long_manual_check"
    if EXCLUDE_RE.search(text):
        return dur, "reject", "excluded_genre_term"
    return dur, "pass", ""


def tier_prelim(item):
    lic = item["status"].get("license")
    if lic == "creativeCommon":
        return "A-check" if item["contentDetails"].get("licensedContent") else "A?"
    return "C?"


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--recheck", action="store_true")
    args = ap.parse_args()
    yt, con = youtube_client(), db()
    where = "1=1" if args.recheck else "meta_checked_at IS NULL"
    ids = [r["video_id"] for r in con.execute(f"SELECT video_id FROM videos WHERE {where}")]
    print(f"{len(ids)} videos to check")
```

```
    try:
        for i in range(0, len(ids), 50):
            batch = ids[i:i + 50]
            resp = yt.videos().list(part="snippet,contentDetails,status",
                                    id=",".join(batch), maxResults=50).execute()
            seen = set()
            for it in resp.get("items", []):
                seen.add(it["id"])
                dur, status, reason = decide(it)
                old = con.execute("SELECT license FROM videos WHERE video_id=?", (it["id"],)).fetchone()
                new_lic = it["status"].get("license")
                if old and old["license"] and old["license"] != new_lic:
                    log_event(con, "video", it["id"], "licence_changed",
                              {"from": old["license"], "to": new_lic})
                sn = it["snippet"]
                con.execute("""UPDATE videos SET channel_id=?, title=?, description=?, published_at=?,
                               duration_s=?, license=?, licensed_content=?, made_for_kids=?,
                               default_audio_lang=?, live=?, privacy=?, meta_checked_at=?,
                               meta_status=?, meta_reason=?, tier_prelim=? WHERE video_id=?""",
                            (sn["channelId"], sn.get("title"), sn.get("description", "")[:5000],
                             sn.get("publishedAt"), dur, new_lic,
                             int(bool(it["contentDetails"].get("licensedContent"))),
                             int(bool(it["status"].get("madeForKids"))),
                             sn.get("defaultAudioLanguage"), sn.get("liveBroadcastContent"),
                             it["status"].get("privacyStatus"), now(), status, reason,
                             tier_prelim(it), it["id"]))
                con.execute("INSERT OR IGNORE INTO channels(channel_id, title, discovered_via, discovered_at)"
                            " VALUES (?,?,?,?)", (sn["channelId"], sn.get("channelTitle"),
                                                  f"video:{it['id']}", now()))
            for gone in set(batch) - seen:        # deleted, private or region-blocked
                con.execute("UPDATE videos SET meta_checked_at=?, meta_status='gone', "
                            "meta_reason='not_returned_by_api' WHERE video_id=?", (now(), gone))
                log_event(con, "video", gone, "gone")
            con.commit()
    except HttpError as err:
        con.commit()
        if is_quota_error(err):
            print("Daily quota exhausted - rerun tomorrow; progress is saved.")
            return
        raise
    for r in con.execute("SELECT meta_status, tier_prelim, COUNT(*) n FROM videos "
                         "GROUP BY meta_status, tier_prelim ORDER BY n DESC"):
        print(dict(r))


if __name__ == "__main__":
    main()
```

D.6 `s04_licence_evidence.py`

```
"""s04_licence_evidence.py - Step 4: capture durable evidence of the licence at collection time.

For every Creative Commons candidate that passed metadata screening, ask the Internet
Archive's Wayback Machine to snapshot the watch page, and record the API licence value with
a timestamp. Tier A material needs this record in case an uploader later switches the video
back to the standard licence.

    python s04_licence_evidence.py --limit 200

The public Save Page Now endpoint is rate limited: keep the default pause, and if you get
HTTP 429 wait an hour. Also take a manual screenshot of the licence line in the description
for every video that ends up in Tier A (save it under licence_evidence/<video_id>.png).
"""
import argparse
import time

import requests

from common import db, log_event, now


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--limit", type=int, default=200)
    ap.add_argument("--pause", type=float, default=15.0, help="seconds between Wayback requests")
    args = ap.parse_args()
    con = db()
```

```
    rows = con.execute("""SELECT v.video_id, v.license FROM videos v
                          WHERE v.meta_status='pass' AND v.tier_prelim LIKE 'A%'
                          AND NOT EXISTS (SELECT 1 FROM licence_evidence e
                                          WHERE e.video_id=v.video_id AND e.wayback_url IS NOT NULL)
                          LIMIT ?""", (args.limit,)).fetchall()
    sess = requests.Session()
    sess.headers["User-Agent"] = "FAST-NUCES FYP Urdu corpus licence evidence (contact: see README)"
    for r in rows:
        url = f"https://www.youtube.com/watch?v={r['video_id']}"
        snap, note = None, ""
        try:
            resp = sess.get(f"https://web.archive.org/save/{url}", timeout=120, allow_redirects=True)
            if "/web/" in resp.url:
                snap = resp.url
            else:
                snap = resp.headers.get("Content-Location")
                snap = f"https://web.archive.org{snap}" if snap else None
            note = f"http {resp.status_code}"
            if resp.status_code == 429:
                print("Rate limited by the Wayback Machine - stopping; rerun later.")
                break
        except requests.RequestException as exc:
            note = f"error: {exc}"
        con.execute("INSERT INTO licence_evidence VALUES (?,?,?,?,?,?)",
                    (r["video_id"], now(), r["license"], snap, None, note))
        log_event(con, "video", r["video_id"], "licence_evidence", {"wayback": snap, "note": note})
        con.commit()
        print(r["video_id"], snap or note)
        time.sleep(args.pause)


if __name__ == "__main__":
    main()
```

D.7 `s05_download.py`

```
"""s05_download.py - Step 5: download the original audio stream and make a 16 kHz working copy.

    python s05_download.py --limit 50                # CC candidates from approved channels first
    python s05_download.py --limit 50 --tier any

For each video this script:
  1. downloads the best audio-only stream with yt-dlp, unchanged (usually Opus in .webm or AAC in .m4a)
  2. records codec / sample rate / channels / bitrate / duration with ffprobe
  3. writes a 16 kHz, mono, 16-bit PCM WAV working copy with ffmpeg
  4. stores SHA-256 checksums of both files
It stops the whole run if YouTube starts asking for sign-in / bot verification.
Never try to get around such a block; slow down and talk to your supervisor instead.
"""
import argparse
import hashlib
import json
import subprocess

from common import RAW_DIR, TARGET_SR, WAV_DIR, db, log_event, now

YTDLP_OPTS = [
    "-f", "bestaudio/best",          # audio-only stream, no re-encoding
    "--no-playlist",
    "--write-info-json",             # keeps title, channel, upload date, licence text, description
    "--no-overwrites",
    "--retries", "10", "--fragment-retries", "10",
    "--sleep-requests", "1",         # pause between metadata requests
    "--sleep-interval", "5", "--max-sleep-interval", "20",   # random pause before each download
    "--print", "after_move:filepath", "--no-simulate",
]
BLOCK_SIGNS = ("Sign in to confirm", "not a bot", "HTTP Error 429")


def sha256(path, chunk=1 << 20):
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for block in iter(lambda: f.read(chunk), b""):
            h.update(block)
    return h.hexdigest()
```

```
def probe(path):
    out = subprocess.run(["ffprobe", "-v", "error", "-select_streams", "a:0",
                          "-show_entries",
"stream=codec_name,sample_rate,channels,bit_rate:format=duration,bit_rate",
                          "-of", "json", str(path)], capture_output=True, text=True, check=True)
    j = json.loads(out.stdout)
    st = j["streams"][0]
    br = st.get("bit_rate") or j["format"].get("bit_rate")
    return (st.get("codec_name"), int(st.get("sample_rate", 0)), int(st.get("channels", 0)),
            int(br) if br and br != "N/A" else None, float(j["format"]["duration"]))


def to_wav(src, dst):
    subprocess.run(["ffmpeg", "-hide_banner", "-nostdin", "-loglevel", "error", "-y",
                    "-i", str(src), "-map", "0:a:0", "-vn",
                    "-ac", "1", "-ar", str(TARGET_SR), "-c:a", "pcm_s16le", str(dst)], check=True)


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--limit", type=int, default=50)
    ap.add_argument("--tier", default="A", choices=["A", "any"])
    ap.add_argument("--include-unvetted", action="store_true",
                    help="also download from channels not yet approved (not recommended)")
    args = ap.parse_args()
    con = db()
    ytdlp_version = subprocess.run(["yt-dlp", "--version"], capture_output=True, text=True).stdout.strip()
    sql = """SELECT v.video_id, v.channel_id FROM videos v JOIN channels c USING(channel_id)
             WHERE v.meta_status='pass' AND v.video_id NOT IN (SELECT video_id FROM downloads
                                                               WHERE status='ok')"""
    if args.tier == "A":
        sql += " AND v.tier_prelim LIKE 'A%'"
    if not args.include_unvetted:
        sql += " AND c.vetting='approved'"
    sql += " ORDER BY v.tier_prelim, v.duration_s DESC LIMIT ?"
    for r in con.execute(sql, (args.limit,)).fetchall():
        vid, ch = r["video_id"], r["channel_id"]
        (RAW_DIR / ch).mkdir(exist_ok=True)
        cmd = ["yt-dlp", *YTDLP_OPTS, "-o", str(RAW_DIR / ch / "%(id)s.%(ext)s"),
               f"https://www.youtube.com/watch?v={vid}"]
        res = subprocess.run(cmd, capture_output=True, text=True)
        if any(sig in res.stderr for sig in BLOCK_SIGNS):
            con.execute("INSERT OR REPLACE INTO downloads(video_id, downloaded_at, status, error, ytdlp_version)"
                        " VALUES (?,?,?,?,?)", (vid, now(), "blocked", res.stderr[-500:], ytdlp_version))
            con.commit()
            raise SystemExit("YouTube is asking for verification. Stop for today, lower the rate, "
                             "and do NOT attempt to bypass it (see Section 7.5).")
        paths = [p for p in res.stdout.strip().splitlines() if p]
        if res.returncode != 0 or not paths:
            con.execute("INSERT OR REPLACE INTO downloads(video_id, downloaded_at, status, error, ytdlp_version)"
                        " VALUES (?,?,?,?,?)", (vid, now(), "failed", res.stderr[-500:], ytdlp_version))
            log_event(con, "video", vid, "download_failed", res.stderr[-300:])
            con.commit()
            continue
        orig = paths[-1]
        wav = WAV_DIR / f"{vid}.wav"
        try:
            codec, sr, chans, br, dur = probe(orig)
            to_wav(orig, wav)
        except subprocess.CalledProcessError as exc:
            con.execute("INSERT OR REPLACE INTO downloads(video_id, downloaded_at, orig_path, status, error,"
                        " ytdlp_version) VALUES (?,?,?,?,?,?)",
                        (vid, now(), orig, "convert_failed", str(exc)[-500:], ytdlp_version))
            con.commit()
            continue
        con.execute("""INSERT OR REPLACE INTO downloads VALUES (?,?,?,?,?,?,?,?,?,?,?,?,?)""",
                    (vid, now(), orig, codec, sr, chans, br, dur, str(wav),
                     sha256(orig), sha256(wav), ytdlp_version, "ok", None))
        log_event(con, "video", vid, "downloaded", {"codec": codec, "sr": sr, "duration": dur})
        con.commit()
        print(f"ok {vid} {codec} {sr}Hz {chans}ch {dur / 60:.1f} min")


if __name__ == "__main__":
    main()
```

D.8 `s06_auto_screen.py`

```
"""s06_auto_screen.py - Step 6: automatic audio screening and ranking (GPU recommended).

Two stages keep compute manageable:
    python s06_auto_screen.py --stage probe --limit 100   # 3 x 5-minute probes per video (cheap)
    python s06_auto_screen.py --stage full  --min-score 0.45 --limit 40   # whole file, best probes only

Per video it measures:
  speech / music / noise proportions ........ inaSpeechSegmenter
  Urdu(+Hindi) vs English vs other language .. Whisper language ID on 30 s speech windows
  speakers, overlap, max concurrency, turns .. pyannote speaker diarization
and combines them into a 0-1 ranking score. The full-stage RTTM is kept: it becomes the
machine pre-annotation for the annotation stage.
"""
import argparse
import math
import os

import numpy as np
import soundfile as sf

from common import (HF_TOKEN, PROBE_DIR, RTTM_DIR, TARGET_SR, db, log_event, now, speaker_changes,
                    speech_stats, write_rttm)

DIAR_MODEL = os.environ.get("DIAR_MODEL", "pyannote/speaker-diarization-3.1")
LID_MODEL = os.environ.get("LID_MODEL", "small")
URDU_LIKE = {"ur", "hi"}   # spoken Urdu and Hindi are close; Whisper often labels Urdu speech as "hi"


def load_models():
    import torch
    import whisper
    from inaSpeechSegmenter import Segmenter
    from pyannote.audio import Pipeline
    dev = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    try:
        diar = Pipeline.from_pretrained(DIAR_MODEL, token=HF_TOKEN)             # pyannote.audio 4.x
    except TypeError:
        diar = Pipeline.from_pretrained(DIAR_MODEL, use_auth_token=HF_TOKEN)    # pyannote.audio 3.x
    diar.to(dev)
    return Segmenter(vad_engine="smn", detect_gender=False), whisper.load_model(LID_MODEL, device=dev), diar


def make_probe(wav, vid, minutes=5):
    """Concatenate three 5-minute probes (10 %, 50 %, 80 % into the file) into one temporary WAV."""
    info = sf.info(str(wav))
    n = int(minutes * 60 * info.samplerate)
    parts = []
    for frac in (0.10, 0.50, 0.80):
        start = max(0, min(int(info.frames * frac), info.frames - n))
        data, _ = sf.read(str(wav), start=start, frames=n, dtype="int16")
        parts.append(data)
    out = PROBE_DIR / f"{vid}.probe.wav"
    sf.write(str(out), np.concatenate(parts), info.samplerate, subtype="PCM_16")
    return out


def ina_ratios(segmenter, path, total):
    seg = segmenter(str(path))                      # list of (label, start, end)
    dur = {"speech": 0.0, "music": 0.0, "noise": 0.0}
    speech_regions = []
    for label, s, e in seg:
        if label in dur:
            dur[label] += e - s
        if label == "speech":
            speech_regions.append((s, e))
    return {k: v / total for k, v in dur.items()}, speech_regions


def lid_fractions(model, path, speech_regions, win=30.0, step=120.0):
    import whisper
    audio = whisper.load_audio(str(path))           # float32, 16 kHz
    counts, t, total = {}, 0.0, len(audio) / TARGET_SR
    while t + win <= total:
        covered = sum(max(0.0, min(e, t + win) - max(s, t)) for s, e in speech_regions)
        if covered >= 0.7 * win:                    # only judge windows that are mostly speech
            seg = whisper.pad_or_trim(audio[int(t * TARGET_SR):int((t + win) * TARGET_SR)])
```

```
            mel = whisper.log_mel_spectrogram(seg, n_mels=model.dims.n_mels).to(model.device)
            _, probs = model.detect_language(mel)
            top = max(probs, key=probs.get)
            counts[top] = counts.get(top, 0) + 1
        t += step
    n = sum(counts.values())
    if n == 0:
        return 0.0, 0.0, 0.0, 0
    ur = sum(counts.get(k, 0) for k in URDU_LIKE) / n
    en = counts.get("en", 0) / n
    return ur, en, 1.0 - ur - en, n


def diarize(pipe, path):
    out = pipe(str(path))
    ann = getattr(out, "speaker_diarization", out)   # 4.x wraps the annotation in an output object
    return [(turn.start, turn.end, spk) for turn, _, spk in ann.itertracks(yield_label=True)]


def score(m):
    """Ranking score in [0, 1]. No upper limit on speakers: more speakers never lowers the score."""
    spk = min(math.log2(max(m["n_speakers"], 1)) / math.log2(8), 1.0)   # 1 spk->0, 2->0.33, 8+->1
    ovl = min(m["overlap_ratio"] / 0.15, 1.0)                           # 15 % overlap saturates
    s = (0.30 * ovl + 0.25 * spk + 0.25 * m["urdu_hindi_frac"] + 0.20 * m["speech_ratio"]
         - 0.40 * m["music_ratio"])
    if m["n_speakers"] < 2:
        s = 0.0                                                         # single speaker: not usable
    return round(max(0.0, min(1.0, s)), 4)


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--stage", choices=["probe", "full"], required=True)
    ap.add_argument("--limit", type=int, default=100)
    ap.add_argument("--min-score", type=float, default=0.45, help="full stage: probe score needed")
    args = ap.parse_args()
    con = db()
    if args.stage == "probe":
        rows = con.execute("""SELECT d.video_id, d.wav_path, d.duration_s FROM downloads d
                              WHERE d.status='ok' AND d.video_id NOT IN
                              (SELECT video_id FROM screening WHERE stage='probe') LIMIT ?""",
                           (args.limit,)).fetchall()
    else:
        rows = con.execute("""SELECT d.video_id, d.wav_path, d.duration_s FROM downloads d
                              JOIN screening s ON s.video_id=d.video_id AND s.stage='probe'
                              WHERE s.score >= ? AND d.video_id NOT IN
                              (SELECT video_id FROM screening WHERE stage='full')
                              ORDER BY s.score DESC LIMIT ?""", (args.min_score, args.limit)).fetchall()
    if not rows:
        print("nothing to screen")
        return
    segmenter, lid, diar = load_models()
    for r in rows:
        vid = r["video_id"]
        path = make_probe(r["wav_path"], vid) if args.stage == "probe" else r["wav_path"]
        total = sf.info(str(path)).duration
        ratios, speech_regions = ina_ratios(segmenter, path, total)
        ur, en, other, n_win = lid_fractions(lid, path, speech_regions)
        segs = diarize(diar, path)
        speech, overlap, max_c = speech_stats(segs)
        m = dict(speech_ratio=ratios["speech"], music_ratio=ratios["music"], noise_ratio=ratios["noise"],
                 urdu_hindi_frac=ur, english_frac=en, other_lang_frac=other, lid_windows=n_win,
                 n_speakers=len({s[2] for s in segs}),
                 overlap_ratio=(overlap / speech) if speech else 0.0, max_concurrent=max_c,
                 changes_per_min=speaker_changes(segs) / (speech / 60) if speech else 0.0)
        rttm = None
        if args.stage == "full":
            rttm = RTTM_DIR / f"{vid}.rttm"
            write_rttm(rttm, vid, segs)
        else:
            os.remove(path)                                  # probes are temporary
        m["score"] = score(m)
        con.execute("""INSERT OR REPLACE INTO screening VALUES (?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?)""",
                    (vid, args.stage, now(), m["speech_ratio"], m["music_ratio"], m["noise_ratio"],
                     m["urdu_hindi_frac"], m["english_frac"], m["other_lang_frac"], m["lid_windows"],
                     m["n_speakers"], m["overlap_ratio"], m["max_concurrent"], m["changes_per_min"],
                     str(rttm) if rttm else None, DIAR_MODEL, m["score"]))
        log_event(con, "video", vid, f"screened_{args.stage}", m)
        con.commit()
```

```
        print(f"{vid} [{args.stage}] spk={m['n_speakers']} ovl={m['overlap_ratio']:.1%} "
              f"urdu={ur:.0%} music={ratios['music']:.0%} score={m['score']}")


if __name__ == "__main__":
    main()
```

D.9 `s07_triage.py`

```
"""s07_triage.py - Step 7: human triage through a shared spreadsheet.

    python s07_triage.py export --top 150 --out triage_batch_03.csv
        -> open in Google Sheets / Excel, split rows between reviewers, fill the empty columns
    python s07_triage.py import triage_batch_03_done.csv

Allowed values (anything else is rejected on import):
  decision:          accept | reject | second_opinion
  tier:              A | B | C | X        (see Section 3.8)
  yes/no columns:    yes | no | unsure
  format_type:       podcast | talk_show | panel | interview | debate | vlog_group | classroom | other
"""
import argparse
import csv
import sys

from common import db, log_event, now

YN = {"yes", "no", "unsure"}
FILL = ["reviewer", "decision", "tier", "original_creator", "conversational", "urdu_dominant",
        "music_bed", "inserted_clips", "minors", "private_info", "sensitive", "format_type", "notes"]
ALLOWED = {
    "decision": {"accept", "reject", "second_opinion"},
    "tier": {"A", "B", "C", "X"},
    "original_creator": YN, "conversational": YN, "urdu_dominant": YN, "music_bed": YN,
    "inserted_clips": YN, "minors": YN, "private_info": YN, "sensitive": YN,
    "format_type": {"podcast", "talk_show", "panel", "interview", "debate", "vlog_group",
                    "classroom", "other"},
}


def export(con, top, out):
    rows = con.execute("""SELECT v.video_id, v.title, c.title AS channel, v.license, v.tier_prelim,
                                 v.duration_s, s.score, s.n_speakers, s.overlap_ratio, s.urdu_hindi_frac,
                                 s.music_ratio, s.stage
                          FROM screening s JOIN videos v USING(video_id)
                          JOIN channels c ON c.channel_id = v.channel_id
                          WHERE s.stage = 'full'
                            AND v.video_id NOT IN (SELECT video_id FROM triage WHERE decision != 'second_opinion')
                          ORDER BY s.score DESC LIMIT ?""", (top,)).fetchall()
    with open(out, "w", newline="", encoding="utf-8-sig") as f:      # utf-8-sig so Excel shows Urdu
        w = csv.writer(f)
        w.writerow(["video_id", "url", "title", "channel", "api_license", "tier_prelim", "minutes",
                    "score", "hyp_speakers", "hyp_overlap", "urdu_frac", "music_frac", "stage"] + FILL)
        for r in rows:
            w.writerow([r["video_id"], f"https://www.youtube.com/watch?v={r['video_id']}", r["title"],
                        r["channel"], r["license"], r["tier_prelim"], round((r["duration_s"] or 0) / 60, 1),
                        r["score"], r["n_speakers"], f"{r['overlap_ratio']:.1%}",
                        f"{r['urdu_hindi_frac']:.0%}", f"{r['music_ratio']:.0%}", r["stage"]]
                       + [""] * len(FILL))
    print(f"wrote {len(rows)} rows to {out}")


def import_(con, path):
    errors, ok = [], 0
    with open(path, encoding="utf-8-sig") as f:
        for i, row in enumerate(csv.DictReader(f), start=2):
            if not row.get("decision"):
                continue                                          # not reviewed yet
            vals = {k: (row.get(k) or "").strip() for k in FILL}
            for k, allowed in ALLOWED.items():
                if vals[k] and vals[k] not in allowed:
                    errors.append(f"line {i}: {k}={vals[k]!r} not in {sorted(allowed)}")
            if vals["decision"] == "accept" and vals["tier"] == "A" and vals["original_creator"] != "yes":
                errors.append(f"line {i}: Tier A requires original_creator=yes")
            if vals["decision"] == "accept" and "yes" in (vals["minors"], vals["private_info"]):
                errors.append(f"line {i}: accepted despite minors/private_info=yes - needs second_opinion")
            if not vals["reviewer"]:
```

```
                errors.append(f"line {i}: reviewer missing")
            if errors and errors[-1].startswith(f"line {i}:"):
                continue
            con.execute("INSERT INTO triage VALUES (?,?,?,?,?,?,?,?,?,?,?,?,?,?,?)",
                        (row["video_id"], vals["reviewer"], now(), vals["decision"], vals["tier"],
                         vals["original_creator"], vals["conversational"], vals["urdu_dominant"],
                         vals["music_bed"], vals["inserted_clips"], vals["minors"], vals["private_info"],
                         vals["sensitive"], vals["format_type"], vals["notes"]))
            log_event(con, "video", row["video_id"], f"triage:{vals['decision']}", vals, actor=vals["reviewer"])
            ok += 1
    con.commit()
    print(f"imported {ok} decisions")
    if errors:
        print("\n".join(errors), file=sys.stderr)
        sys.exit(1)


def main():
    ap = argparse.ArgumentParser()
    sub = ap.add_subparsers(dest="cmd", required=True)
    e = sub.add_parser("export")
    e.add_argument("--top", type=int, default=150)
    e.add_argument("--out", required=True)
    i = sub.add_parser("import")
    i.add_argument("path")
    args = ap.parse_args()
    con = db()
    export(con, args.top, args.out) if args.cmd == "export" else import_(con, args.path)


if __name__ == "__main__":
    main()
```

D.10 `s08_select_excerpts.py`

```
"""s08_select_excerpts.py - Step 8: choose 10-15 minute excerpts to send to annotation.

    python s08_select_excerpts.py --target-min 12 --per-video 2 --channel-cap-hours 3

For every video accepted in triage (Tier A, B or C) it slides a window over the machine
diarization, scores each window (overlap, number of speakers, speech density), keeps the best
non-overlapping windows, and moves each boundary to the nearest pause so no turn is cut in half.
A per-channel cap stops one prolific channel from dominating the corpus.
"""
import argparse
import math

from common import db, log_event, now, read_rttm, speech_stats, speech_union


def window_score(segs, s, e):
    inside = [(max(a, s), min(b, e), k) for a, b, k in segs if b > s and a < e]
    speech, overlap, _ = speech_stats(inside)
    n_spk = len({k for _, _, k in inside})
    ovl = min((overlap / speech if speech else 0) / 0.15, 1.0)
    spk = min(math.log2(max(n_spk, 1)) / math.log2(8), 1.0)
    return 0.5 * ovl + 0.3 * spk + 0.2 * (speech / (e - s)), n_spk, (overlap / speech if speech else 0)


def cut_points(segs, duration, min_gap=0.4):
    """Mid-points of pauses of at least min_gap seconds (nobody speaking), plus file start/end."""
    pts, prev_end = [0.0], 0.0
    for s, e in speech_union(segs):
        if s - prev_end >= min_gap:
            pts.append((prev_end + s) / 2)
        prev_end = e
    pts.append(duration)
    return pts


def snap(t, pts, tol=45.0):
    best = min(pts, key=lambda p: abs(p - t))
    return (best, True) if abs(best - t) <= tol else (t, False)


def main():
    ap = argparse.ArgumentParser()
```

```
    ap.add_argument("--target-min", type=float, default=12)
    ap.add_argument("--min-min", type=float, default=8)
    ap.add_argument("--max-min", type=float, default=15)
    ap.add_argument("--per-video", type=int, default=2)
    ap.add_argument("--channel-cap-hours", type=float, default=3.0)
    ap.add_argument("--step-s", type=float, default=30)
    args = ap.parse_args()
    con = db()
    rows = con.execute("""
        SELECT v.video_id, v.channel_id, d.duration_s, s.rttm_path, s.score, t.tier
        FROM triage t
        JOIN (SELECT video_id, MAX(reviewed_at) m FROM triage GROUP BY video_id) last
             ON last.video_id = t.video_id AND last.m = t.reviewed_at
        JOIN videos v ON v.video_id = t.video_id
        JOIN downloads d ON d.video_id = t.video_id
        JOIN screening s ON s.video_id = t.video_id AND s.stage = 'full'
        WHERE t.decision = 'accept' AND t.tier IN ('A','B','C')
          AND t.video_id NOT IN (SELECT video_id FROM excerpts)
        ORDER BY s.score DESC""").fetchall()
    used = {r["channel_id"]: r["h"] for r in con.execute(
        "SELECT channel_id, SUM(end_s - start_s)/3600.0 h FROM excerpts GROUP BY channel_id")}
    L = args.target_min * 60
    made = 0
    for r in rows:
        segs, dur = read_rttm(r["rttm_path"]), r["duration_s"]
        if not segs or dur < args.min_min * 60:
            continue
        if dur <= args.max_min * 60:
            candidates = [(window_score(segs, 0, dur), 0.0, dur)]
        else:
            candidates, t = [], 0.0
            while t + L <= dur:
                candidates.append((window_score(segs, t, t + L), t, t + L))
                t += args.step_s
        candidates.sort(key=lambda c: c[0][0], reverse=True)
        pts, chosen = cut_points(segs, dur), []
        for (sc, n_spk, ovl), s, e in candidates:
            if len(chosen) >= args.per_video:
                break
            if used.get(r["channel_id"], 0) >= args.channel_cap_hours:
                break
            if any(s < ce + 60 and e > cs - 60 for cs, ce in chosen):    # keep excerpts apart
                continue
            s2, ok_s = snap(s, pts)
            e2, ok_e = snap(e, pts)
            if not (args.min_min * 60 <= e2 - s2 <= args.max_min * 60 + 90):
                s2, e2 = s, e
            chosen.append((s2, e2))
            ex_id = f"{r['video_id']}__{int(round(s2 * 1000)):09d}"
            con.execute("INSERT OR REPLACE INTO excerpts VALUES (?,?,?,?,?,?,?,?,?,?,?)",
                        (ex_id, r["video_id"], r["channel_id"], round(s2, 3), round(e2, 3),
                         n_spk, round(ovl, 4), round(sc, 4), r["tier"], None, now()))
            log_event(con, "excerpt", ex_id, "selected",
                      {"start": s2, "end": e2, "snapped": [ok_s, ok_e], "score": sc})
            used[r["channel_id"]] = used.get(r["channel_id"], 0) + (e2 - s2) / 3600
            made += 1
        con.commit()
    total = con.execute("SELECT COUNT(*) n, SUM(end_s-start_s)/3600.0 h FROM excerpts").fetchone()
    print(f"selected {made} new excerpts; corpus now {total['n']} excerpts, {total['h'] or 0:.1f} h")


if __name__ == "__main__":
    main()
```

D.11 `s09_assign_splits.py`

```
"""s09_assign_splits.py - Step 9a: channel-disjoint train / dev / test assignment.

    python s09_assign_splits.py --test 0.10 --dev 0.10 --seed 2026

Whole channels go to exactly one split, so a podcast host can never appear in both training
and test data. Channels that already have a split keep it (assignments are frozen once made);
new channels are used to top up whichever split is furthest below its target.
Dev/test prefer redistributable channels (Tier A/B) so the evaluation sets do not decay when
YouTube videos disappear.
"""
import argparse
```

```
import random

from common import db, log_event


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--test", type=float, default=0.10)
    ap.add_argument("--dev", type=float, default=0.10)
    ap.add_argument("--seed", type=int, default=2026)
    args = ap.parse_args()
    con = db()
    rows = con.execute("""SELECT channel_id, SUM(end_s-start_s)/3600.0 h,
                                 MIN(tier) best_tier, MAX(split) split
                          FROM excerpts GROUP BY channel_id""").fetchall()
    total = sum(r["h"] for r in rows)
    have = {"train": 0.0, "dev": 0.0, "test": 0.0}
    new = []
    for r in rows:
        if r["split"]:
            have[r["split"]] += r["h"]
        else:
            new.append(r)
    target = {"test": args.test * total, "dev": args.dev * total}
    rng = random.Random(args.seed)
    rng.shuffle(new)
    new.sort(key=lambda r: r["best_tier"] not in ("A", "B"))   # redistributable channels first (stable sort)
    for r in new:
        split = "train"
        for s in ("test", "dev"):
            deficit = target[s] - have[s]
            if deficit > 0 and r["h"] <= max(deficit * 2, 0.5):     # stop one big channel overshooting
                split = s
                break
        have[split] += r["h"]
        con.execute("UPDATE excerpts SET split=? WHERE channel_id=?", (split, r["channel_id"]))
        log_event(con, "channel", r["channel_id"], "split_assigned", {"split": split, "hours": r["h"]})
    con.commit()
    for s, h in have.items():
        print(f"{s:5s} {h:6.2f} h  ({h / total:.0%})" if total else s)


if __name__ == "__main__":
    main()
```

D.12 `s10_speaker_link.py`

```
"""s10_speaker_link.py - Step 9b: flag the same person appearing in different splits.

    python s10_speaker_link.py --threshold 0.55 --out speaker_link_review.csv

Channel-disjoint splits still leak when a guest, anchor or politician appears on several
channels. For every excerpt this script embeds each machine-hypothesised speaker (up to 30 s
of their longest non-overlapped turns), compares speakers across different splits with cosine
similarity, and writes pairs above the threshold to a CSV for a human to listen to and resolve
(move a channel, or drop the excerpt). The threshold is a starting point: calibrate it on a few
known same/different pairs from your pilot.
"""
import argparse
import csv
import os

import numpy as np

from common import EMB_DIR, HF_TOKEN, db, read_rttm, speech_stats

EMB_MODEL = os.environ.get("EMB_MODEL", "pyannote/wespeaker-voxceleb-resnet34-LM")


def load_inference():
    from pyannote.audio import Inference, Model
    try:
        model = Model.from_pretrained(EMB_MODEL, token=HF_TOKEN)
    except TypeError:
        model = Model.from_pretrained(EMB_MODEL, use_auth_token=HF_TOKEN)
    return Inference(model, window="whole")
```

```
def clean_turns(segs, spk, s0, e0):
    """Turns of one speaker inside the excerpt that overlap no-one else, longest first."""
    mine = [(max(a, s0), min(b, e0)) for a, b, k in segs if k == spk and b > s0 and a < e0]
    others = [(a, b, k) for a, b, k in segs if k != spk]
    clean = [(a, b) for a, b in mine
             if b - a >= 1.5 and speech_stats([(a, b, spk)] + [o for o in others if o[1] > a and o[0] < b])[1] ==
0]
    return sorted(clean, key=lambda t: t[1] - t[0], reverse=True)


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--threshold", type=float, default=0.55)
    ap.add_argument("--out", default="speaker_link_review.csv")
    args = ap.parse_args()
    from pyannote.core import Segment
    con = db()
    inference = load_inference()
    rows = con.execute("""SELECT e.excerpt_id, e.video_id, e.split, e.start_s, e.end_s, d.wav_path, s.rttm_path
                          FROM excerpts e JOIN downloads d USING(video_id)
                          JOIN screening s ON s.video_id=e.video_id AND s.stage='full'
                          WHERE e.split IS NOT NULL""").fetchall()
    items = []                                  # (excerpt_id, split, speaker, vector)
    for r in rows:
        cache = EMB_DIR / f"{r['excerpt_id']}.npz"
        if cache.exists():
            z = np.load(cache, allow_pickle=True)
            items += [(r["excerpt_id"], r["split"], k, z[k]) for k in z.files]
            continue
        segs, vecs = read_rttm(r["rttm_path"]), {}
        for spk in sorted({k for _, _, k in segs}):
            turns, got, parts = clean_turns(segs, spk, r["start_s"], r["end_s"]), 0.0, []
            for a, b in turns:
                if got >= 30:
                    break
                parts.append(np.asarray(inference.crop(r["wav_path"], Segment(a, b))).reshape(-1))
                got += b - a
            if parts:
                v = np.mean(parts, axis=0)
                vecs[spk] = v / np.linalg.norm(v)
        np.savez(cache, **vecs)
        items += [(r["excerpt_id"], r["split"], k, v) for k, v in vecs.items()]
    flagged = []
    if items:
        mat = np.stack([it[3] for it in items])            # unit vectors -> dot product = cosine
        splits = np.array([it[1] for it in items])
        sims = mat @ mat.T
        ii, jj = np.where(np.triu(sims >= args.threshold, k=1) & (splits[:, None] != splits[None, :]))
        for i, j in zip(ii, jj):
            a, b = items[i], items[j]
            flagged.append((round(float(sims[i, j]), 3), a[0], a[1], a[2], b[0], b[1], b[2]))
    flagged.sort(reverse=True)
    with open(args.out, "w", newline="", encoding="utf-8") as f:
        w = csv.writer(f)
        w.writerow(["similarity", "excerpt_a", "split_a", "speaker_a", "excerpt_b", "split_b", "speaker_b",
                    "same_person(yes/no)", "resolution", "reviewer"])
        w.writerows(flagged)
    print(f"{len(items)} speaker embeddings, {len(flagged)} cross-split pairs to review -> {args.out}")


if __name__ == "__main__":
    main()
```

D.13 `s11_package.py`

```
"""s11_package.py - Step 10: build the hand-off package for the annotation/transcription stage.

    python s11_package.py --version v0.1

Creates handoff/<version>/ containing sample-exact excerpt WAVs, machine pre-annotation RTTMs,
UEM scoring regions, split lists, a pyannote.database config, manifests, a source/licence table,
ATTRIBUTION.md, public_manifest.csv (what may be redistributed) and SHA-256 checksums.
The package is internal: only the files listed in public_manifest.csv may ever be published.
"""
import argparse
import csv
```

```
import hashlib
import json

import soundfile as sf

from common import HANDOFF_DIR, TARGET_SR, db, log_event, now, read_rttm, write_rttm

CC_BY_3 = "https://creativecommons.org/licenses/by/3.0/"


def sha256(path):
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for b in iter(lambda: f.read(1 << 20), b""):
            h.update(b)
    return h.hexdigest()


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--version", required=True)
    args = ap.parse_args()
    con = db()
    out = HANDOFF_DIR / args.version
    for sub in ("audio", "rttm_machine", "uem", "lists"):
        (out / sub).mkdir(parents=True, exist_ok=True)
    rows = con.execute("""
        SELECT e.*, d.wav_path, s.rttm_path, s.diar_model, v.title, v.published_at, v.license,
               c.title AS channel_title,
               (SELECT wayback_url FROM licence_evidence le WHERE le.video_id=e.video_id
                  AND le.wayback_url IS NOT NULL ORDER BY checked_at DESC LIMIT 1) AS wayback
        FROM excerpts e JOIN downloads d USING(video_id) JOIN videos v USING(video_id)
        JOIN channels c ON c.channel_id = e.channel_id
        JOIN screening s ON s.video_id = e.video_id AND s.stage = 'full'
        WHERE e.split IS NOT NULL ORDER BY e.split, e.excerpt_id""").fetchall()
    manifest, per_split = [], {"train": [], "dev": [], "test": []}
    split_rttm = {k: [] for k in per_split}
    for r in rows:
        ex = r["excerpt_id"]
        info = sf.info(r["wav_path"])
        assert info.samplerate == TARGET_SR and info.channels == 1, f"unexpected format for {ex}"
        a, b = int(round(r["start_s"] * TARGET_SR)), int(round(r["end_s"] * TARGET_SR))
        audio, _ = sf.read(r["wav_path"], start=a, stop=b, dtype="int16")
        wav_out = out / "audio" / f"{ex}.wav"
        sf.write(str(wav_out), audio, TARGET_SR, subtype="PCM_16")
        dur = len(audio) / TARGET_SR
        # machine hypotheses clipped to the excerpt, shifted to start at 0, speakers renamed per excerpt
        segs = [(max(s, r["start_s"]) - r["start_s"], min(e, r["end_s"]) - r["start_s"], k)
                for s, e, k in read_rttm(r["rttm_path"]) if e > r["start_s"] and s < r["end_s"]]
        names = {k: f"spk{i:02d}" for i, k in enumerate(sorted({k for *_, k in segs}))}
        segs = [(s, e, names[k]) for s, e, k in segs if e - s > 0.01]
        write_rttm(out / "rttm_machine" / f"{ex}.rttm", ex, segs)
        split_rttm[r["split"]] += [(ex, segs)]
        (out / "uem" / f"{ex}.uem").write_text(f"{ex} 1 0.000 {dur:.3f}\n")
        per_split[r["split"]].append((ex, dur))
        manifest.append({
            "excerpt_id": ex, "split": r["split"], "tier": r["tier"],
            "audio": f"audio/{ex}.wav", "sample_rate": TARGET_SR, "channels": 1, "sample_format": "s16le",
            "num_samples": len(audio), "duration_s": round(dur, 3), "sha256": sha256(wav_out),
            "source": {"platform": "youtube", "video_id": r["video_id"],
                       "url": f"https://www.youtube.com/watch?v={r['video_id']}",
                       "channel_id": r["channel_id"], "channel_title": r["channel_title"],
                       "title": r["title"], "published_at": r["published_at"],
                       "offset_start_s": r["start_s"], "offset_end_s": r["end_s"],
                       "api_license": r["license"], "licence_evidence": r["wayback"]},
            "machine_preannotation": {"rttm": f"rttm_machine/{ex}.rttm", "model": r["diar_model"],
                                      "n_speakers_hyp": len(names), "overlap_ratio_hyp": r["overlap_ratio_hyp"]},
            "status": {"diarization_gold": False, "transcription": False},
        })
    for split, items in per_split.items():
        (out / "lists" / f"{split}.lst").write_text("".join(f"{ex}\n" for ex, _ in items))
        (out / "uem" / f"{split}.uem").write_text("".join(f"{ex} 1 0.000 {d:.3f}\n" for ex, d in items))
        with open(out / "rttm_machine" / f"{split}.rttm", "w", encoding="utf-8") as f:
            for ex, segs in split_rttm[split]:
                for s, e, k in segs:
                    f.write(f"SPEAKER {ex} 1 {s:.3f} {e - s:.3f} <NA> <NA> {k} <NA> <NA>\n")
    (out / "database.yml").write_text(
        "Databases:\n  UrduYT: audio/{uri}.wav\n\nProtocols:\n  UrduYT:\n    SpeakerDiarization:\n"
        "      MachinePreannotation:\n" + "".join(
```

```
            f"        {name}:\n          uri: lists/{s}.lst\n          annotation: rttm_machine/{s}.rttm\n"
            f"          annotated: uem/{s}.uem\n"
            for name, s in (("train", "train"), ("development", "dev"), ("test", "test"))))
    with open(out / "manifest.jsonl", "w", encoding="utf-8") as f:
        for m in manifest:
            f.write(json.dumps(m, ensure_ascii=False) + "\n")
    with open(out / "sources.csv", "w", newline="", encoding="utf-8-sig") as f:
        w = csv.writer(f)
        w.writerow(["excerpt_id", "split", "tier", "video_id", "url", "channel_title", "title",
                    "start_s", "end_s", "api_license", "licence_evidence"])
        for m in manifest:
            s = m["source"]
            w.writerow([m["excerpt_id"], m["split"], m["tier"], s["video_id"], s["url"], s["channel_title"],
                        s["title"], s["offset_start_s"], s["offset_end_s"], s["api_license"],
                        s["licence_evidence"]])
    with open(out / "public_manifest.csv", "w", newline="", encoding="utf-8") as f:
        w = csv.writer(f)
        w.writerow(["excerpt_id", "video_id", "start_s", "end_s", "audio_redistributable", "licence"])
        for m in manifest:
            s = m["source"]
            redis = m["tier"] in ("A", "B")
            lic = "CC BY 3.0" if m["tier"] == "A" else ("written permission" if m["tier"] == "B" else "reference
only")
            w.writerow([m["excerpt_id"], s["video_id"], s["offset_start_s"], s["offset_end_s"], redis, lic])
    with open(out / "ATTRIBUTION.md", "w", encoding="utf-8") as f:
        f.write("# Attribution\n\nAudio excerpts in this corpus that are marked redistributable were derived "
                "(cut and resampled to 16 kHz mono) from the following works.\n\n")
        for m in manifest:
            if m["tier"] == "A":
                s = m["source"]
                f.write(f"- \"{s['title']}\" by {s['channel_title']} ({s['url']}), licensed under "
                        f"CC BY 3.0 ({CC_BY_3}). Excerpt {s['offset_start_s']:.1f}-{s['offset_end_s']:.1f} s; "
                        f"changes: excerpted, down-mixed to mono, resampled to 16 kHz.\n")
            elif m["tier"] == "B":
                s = m["source"]
                f.write(f"- \"{s['title']}\" by {s['channel_title']} ({s['url']}), used with written "
                        f"permission of the rights holder.\n")
    with open(out / "checksums.sha256", "w") as f:
        for p in sorted(out.rglob("*")):
            if p.is_file() and p.name != "checksums.sha256":
                f.write(f"{sha256(p)}  {p.relative_to(out)}\n")
    log_event(con, "package", args.version, "built", {"excerpts": len(manifest), "at": now()})
    con.commit()
    hours = {k: sum(d for _, d in v) / 3600 for k, v in per_split.items()}
    print(f"package {out}: {len(manifest)} excerpts; hours per split: "
          + ", ".join(f"{k}={v:.2f}" for k, v in hours.items()))


if __name__ == "__main__":
    main()
```

D.14 `s12_maintenance.py`

```
"""s12_maintenance.py - Step 11: monthly re-check report and takedown handling.

Monthly (after running `python s03_screen_metadata.py --recheck`):
    python s12_maintenance.py report
        -> lists selected excerpts whose source video disappeared or changed licence

When a rights holder or a person in a recording asks for removal:
    python s12_maintenance.py takedown --request-id TD-2026-001 --video VIDEO_ID \
        --requester "channel owner, email on file" --action "excerpts removed, audio deleted"
        -> marks every excerpt from that video as tier X, deletes its local audio and machine RTTM,
           and records the request. Rebuild the hand-off package afterwards.
"""
import argparse
import json
import os

from common import RTTM_DIR, db, log_event, now


def report(con):
    gone = con.execute("""SELECT DISTINCT e.video_id, v.meta_reason FROM excerpts e JOIN videos v USING(video_id)
                          WHERE v.meta_status='gone' AND e.tier != 'X'""").fetchall()
    changed = con.execute("""SELECT entity_id, detail, ts FROM events WHERE action='licence_changed'
                             AND entity_id IN (SELECT video_id FROM excerpts WHERE tier != 'X')""").fetchall()
```

```
    print(f"{len(gone)} excerpted videos no longer available on YouTube:")
    for r in gone:
        print("  ", r["video_id"], r["meta_reason"])
    print(f"{len(changed)} licence changes affecting excerpts:")
    for r in changed:
        d = json.loads(r["detail"])
        print(f"   {r['entity_id']} {d['from']} -> {d['to']} (seen {r['ts']})")
    print("\nTier A material stays usable under the licence recorded at collection time (keep the "
          "evidence); still review each case with your supervisor. Tier C items that are gone can no "
          "longer be re-downloaded by users - consider replacing them.")


def takedown(con, request_id, video, requester, action):
    con.execute("INSERT INTO takedowns VALUES (?,?,?,?,?,?)",
                (request_id, now(), requester, video, action, None))
    con.execute("UPDATE excerpts SET tier='X', split=NULL WHERE video_id=?", (video,))
    row = con.execute("SELECT orig_path, wav_path FROM downloads WHERE video_id=?", (video,)).fetchone()
    removed = []
    for p in ([row["orig_path"], row["wav_path"]] if row else []) + [str(RTTM_DIR / f"{video}.rttm")]:
        if p and os.path.exists(p):
            os.remove(p)
            removed.append(p)
    con.execute("UPDATE downloads SET status='removed_takedown' WHERE video_id=?", (video,))
    con.execute("UPDATE takedowns SET completed_at=? WHERE request_id=?", (now(), request_id))
    log_event(con, "video", video, "takedown", {"request": request_id, "removed_files": removed})
    con.commit()
    print(f"takedown {request_id}: {video} removed ({len(removed)} files). Rebuild the package.")


def main():
    ap = argparse.ArgumentParser()
    sub = ap.add_subparsers(dest="cmd", required=True)
    sub.add_parser("report")
    t = sub.add_parser("takedown")
    t.add_argument("--request-id", required=True)
    t.add_argument("--video", required=True)
    t.add_argument("--requester", required=True)
    t.add_argument("--action", default="excerpts removed, audio deleted")
    args = ap.parse_args()
    con = db()
    if args.cmd == "report":
        report(con)
    else:
        takedown(con, args.request_id, args.video, args.requester, args.action)


if __name__ == "__main__":
    main()
```